Subsymbolic AI

Ibrahim Nasser*

Last updated on April 23, 2026

Contents

1 Introduction

Def 1.1. Artificial intelligence (AI): The capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making

Def 1.2. Symbolic AI: A subfield of Artificial Intelligence (AI) based on the assumption that many aspects of intelligence can be achieved by the manipulation of symbols, combining them into meaningful structures (expressions) and manipulating them (using processes) to produce new expressions.

Def 1.3. Statisitical AI: Remedies the two shortcomings of symbolic AI approaches: that all concepts represented by symbols are crisply defined, and that all aspects of the world are knowable/representable in principle. Statistical AI adopts sophisticated mathematical models of uncertainty and uses them to create more accurate world models and reason about them.

Def 1.4. Subsymbolic AI (a.k.a connectionism/neural AI): A subfield of AI that posits that intelligence is inherently tied to brains, where information is represented by a simple sequence pulses that are processed in parallel via simple calculations realized by neurons, and thus concentrates on neural computing.

Def 1.5. Embodied AI: Posits that intelligence cannot be achieved by reasoning about the state of the world (symbolically, statistically, or sub-symbolically), but must be embodied i.e. situated in the world, equipped with a "body" that can interact with it via sensors and actuators. Here, the main method for realizing intelligent behavior is by learning from the world.

Def 1.6. Reasoning: The process of producing valid arguments and predictive world models. There are three forms of reasoning:

- **deductive reasoning** to produce new knowledge from existing knowledge. *Example:* All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.
- **inductive reasoning** to produce knowledge from perception. *Example:* The sun has risen every morning in recorded history. Therefore, the sun will rise tomorrow.
- **abductive reasoning** to produce explanations for observations and given knowledge. *Example:* The grass is wet this morning. If it rained last night, that would explain it. Therefore, it probably rained.

Def 1.7. Inference: The act or process of reaching a **conclusion** about something from known facts or evidence (jointly called **premises**)

Def 1.8. Formal Logic: The science of deductively valid inferences, meaning arguments whose conclusions necessarily follow from their premises by virtue of their form (their structure), regardless of the topic.

Def 1.9. Agent: An agent is a structure $A := \langle \mathcal{P}, \mathcal{A}, f \rangle$ where:

- $\mathcal{P}$ is a set of percepts
- $\mathcal{A}$ is a set of actions
- f is a function $f : \mathcal{P}^* \to \mathcal{A}$ that maps from percepts to actions.

In other words, an agent is anything that perceives its environment via sensors and acts on it with actuators.

Def 1.10. Performance Measure: A function that evaluates a sequence of environments.

Def 1.11. Rational: An agent is called **rational**, if it chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date.

Def 1.12. PEAS: To design rational agents, we must specify the PEAS components: Performance Measure, Environment, Actuators, and Sensors.

Def 1.13. Environment Types: For an agent a we classify the environment e of a by its **type**, which is one of the following. We call e

1. *fully observable*, iff the a's sensors give it access to the complete state of the environment at any point in time; otherwise, we call it *partially observable*.
2. *deterministic*, iff the next state of the environment is completely determined by the current state and a's actions; otherwise, *stochastic*.
3. *episodic*, iff a's experience is divided into atomic episodes, where it perceives and then performs a single action, and the next episode does not depend on previous ones. Otherwise, *sequential*.
4. *dynamic*, iff the environment can change without an action performed by a; otherwise, *static*. If the environment does not change but a's performance measure does, we call e *semidynamic*.

5. *discrete*, iff the sets of e's state and a's actions are couuntable; otherwise, *continuous*.
6. *single-agent*, iff only a acts on e; otherwise *multi-agent*.

Def 1.14. Reflex: An agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ is called a **reflex agent**, iff it only takes the last percept into account when choosing an action, i.e. $f(p_1, \cdots, p_k) = f(p_k)\ \forall p_1 \cdots p_k \in \mathcal{P}$

Def 1.15. Model-Based Agent: A model-based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a \rangle$ is an agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ whose actions depend on:

- a **world model**: a set $\mathcal{S}$ of possible *states*, and a start state $s_0 \in \mathcal{S}$.
- a **transition model** $\mathcal{T}$ that predicts a new state $\mathcal{T}(s, a)$ from a state s and an action a.
- a **sensor model** S that given a state s and a percept p determines a new state $S(s, p)$.
- an **action function** $a : \mathcal{S} \to \mathcal{A}$ that given a state $s \in \mathcal{S}$ selects the next action $a \in \mathcal{A}$.

> **Note**
>
> If the agent is in state s then it took action a and now perceives p, then the agent state will become $s' = S(p, \mathcal{T}(s, a))$ and accordingly take action $a' = a(s')$.

Def 1.16. Goal Based Agent: A goal based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a, \mathcal{G} \rangle$ is a model based agent with an explicit set of goals. It consists of:

- a set of internal states $\mathcal{S}$ and an initial state $s_0 \in \mathcal{S}$,
- a transition model $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$,
- a state update function $S : \mathcal{P} \times \mathcal{S} \to \mathcal{S}$,
- a set of goals $\mathcal{G}$,
- a goal conditioned action function $a : \mathcal{S} \times \mathcal{G} \to \mathcal{A}$ selecting an action given a state and the goals.

Def 1.17. Utility-Based Agent: A utility-based agent uses a world model along with a utility function that models its preferences among the states of that world. It chooses the action that maximizes the expected utility.

Def 1.18. Learning Agent: A learning agent is an agent that can improve its own behavior through experience. It is composed of four main components:

- the performance element, which selects actions based on the agent's current percepts and represents its existing knowledge or behavior,
- the learning element, which improves the performance element over time by analyzing feedback and identifying how to make better decisions,
- the critic, which evaluates the agent's performance according to a given performance standard and provides feedback to the learning element,
- the problem generator, which suggests new and informative actions or experiences that help the agent explore and learn more effectively.

Def 1.19. State Representation: We call a state representation **atomic**, iff it has no internal structure (black box). However, iff each state is characterized by attributes and their values then the representation is **factored**. A **structured** state representation is when include representations of objects, their properties and relationships.

2 Mathematical Language

Def 2.1. Natural Language: A natural language is any form of spoken or signed means of communication that has evolved naturally in humans through use and repetition without conscious planning or premeditation.

Def 2.2. Domain of Discourse: The set of entities, objects, or concepts that a language, discussion, or formal system refers to. It specifies the scope within which terms, statements, and reasoning apply.

Def 2.3. Jargon (Terminology): A jargon (or terminology) is a set of specialized words or phrases (called technical terms or just terms) relating to concepts from a particular domain of discourse.

Def 2.4. Technical Language: A technical language is a natural language extended by a terminology and (possibly) special idioms, discourse markers, and notations.

Def 2.5. Stylized Language: Mathematicians use a stylized language that uses formulae to represent mathematical objects, uses math idioms for special situations (e.g. "iff", "hence", "let... be..., then..."), and classifies statements by role (e.g., definition, Lemma, Theorem, Proof, Example).

Def 2.6. Mathematical Vernacular: A technical language that you need to master to be successful when moving to a new environment (symbolic AI).

Def 2.7. Formulae: A mathematical formula can be a **mathematical statement** (clause) which can be true or false (e.g. $x > 5, 3 + 5 = 7$), or a **mathematical object** (e.g. $3, n, x^2 + y^2 + z^2, \int_1^0 x^{3/2} dx$)

Observation: mathematical vernacular loves to name objects/statements for precise references. For example: Let $p = 3x^2 + 7x$, then $p^3 + 17p + 1$ is $\cdots$

Moreover, mathematical vernacular aggregates objects/statements for cognitive efficiency, examples:

- $a \in S, b \in S,$ and $c \in S \rightsquigarrow a, b, c \in S$ (object aggregation)
- $i \geq 0$ and $i \leq n \rightsquigarrow 0 \leq i \leq n$ (statement aggregation)

Def 2.8. Sequence: mathematical vernacular uses the concept of sequences instead of lists. Sequences are usually finite, but can be infinite as well.

Def 2.9. Ellipses: mathematical vernacular uses ellipses ($\cdots$) as a constructor for sequences. The meaning of an ellipsis is usually considered "obvious" and left for interpretation by the reader.

ellipses allow to write down large objects easily, examples:

- $1, \cdots, n \rightsquigarrow$ the sequence of natural numbers between 1 and n in order.
- $1, 4, 9, 16, \cdots \rightsquigarrow$ the sequence of squares in orders.
- $e_1, \cdots, e_n \rightsquigarrow$ a sequence of objects e_i for $1 \leq i \leq n$

Mathematical statements come in different epistemic varieties:

- **Definitions**: statements that introduce new global identifier for important objects
- **Assertions**: statements that state properties of mathematical objects
- **Examples**: statements that exhibit a witness for some property
- **Axioms**: statements that characterize the objects of a certain domain of discourse or theory. An axiom (postulate) is a statement about mathematical objects that we *assume* to be true.

Mathematical assertions are pragmatically classified into categories:

- **proofs** are arguments that justify the truth of statements beyond any doubt. Proofs are not really statements, but we sometimes treat them together.
- **proposition** is a statement which is interesting in its own right.
- **lemma** is an easily proved statement which is helpful for proving other propositions and theorems, but is usually not particularly interesting in its own right. A lemma is an intermediate theorem that serves as part of a proof of a larger theorem.
- **theorem** is a more important statement than a proposition which says something definitive on the subject, and often takes more effort to prove than a proposition or lemma. A theorem is a statement about mathematical object that we *know* to be true.

- **corollary** is a quick consequence of a proposition or theorem that was proven rececently. It is a theorem that follows directly from another theorem.
- **conjecture** is a statement that is thought to be provable, but has not been yet. A conjecture that is proven is a theorem

Mathematical statement use abbreviations, examples:

- $\wedge$ and $\vee$ for "and" and "or",
- $\neg$ for "not"
- $\forall x.P \quad (\forall x \in S.P)$ for "condition P holds for all x (in S)"
- $\exists x.P \quad (\exists x \in S.P)$ for "there exists an x (in S) such that the condition P holds"
- $\nexists x.P \quad (\nexists x \in S.P)$ for "there exists no x (in S) such that the condition P holds"
- $\exists^1 x.P (\exists^1 x \in S.P)$ for "there exists one and only x (in S) such that the condition P holds"
- "*iff*" for "if and only if", symbolized by $\Leftrightarrow$
- $\Rightarrow$ for "implies"; we can read $A \Rightarrow B$ as "if A then B"

Def 2.10. Declaration: In mathematical vernacular we call a clause that introduce new identifiers together with some properties a declaration. For example Let $\epsilon, \delta > 0 \cdots$

Def 2.11. Scope: The scope of an identifier is the part of an expression where the reference is valid; that is, where the identifier can be used to refer. In the other parts, the identifier may refer to a different entity, or to nothing at all (unbound)

Observations: definition have *global* scope, declarations have *local* scope.

Def 2.12. Definiendum: The definiendum is the symbol, expression, or term being introduced by a definition. It is the newly named object or concept.

Def 2.13. Definiens: The definiens is the symbol, expression, or construction that specifies the meaning of the defineiendum. It explains what the defineiendum denotes.

Mathematics has developed various forms of definition (definition schemata). A **simple definition** introduces a name (the defineiendum) for a compound object or concept (the definiens). The defineiendum must be new, i.e. may not have been used for anything else, in particular, the defineiendum may not occur in the definiens. We use the symbols $:=$ (and the inverse $=:$) to write simple definitions. For example, we can give the natural number 2 the name ϕ. In a formula we write this as $\phi := 2$ or $2 =: \phi$.

On the other hand, in a **pattern definition** the defineiendum is the operator or relation applied to n distinct variables -called pattern variables- $v_1, \cdots, v_n$ as arguments, and the definiens is an expression in these variables. When the new operator is applied to arguments $a_1, \cdots, a_n$, then its value is the definiens expression where the v_i are replaced by the a_i. We use the symbol $:=$ for operator definitions and $:\Leftrightarrow$ for relation definitions. For example, the following is a pattern definition for the set intersection operator $\cap$:

$$A \cap B := \{x \mid x \in A \wedge x \in B\}$$

The pattern variables are A and B, and with this definition we have e.g. $\Phi \cap \Phi = \{x \mid x \in \Phi \wedge x \in \Phi\}$.

An **implicit definition** (definition by description) is a clause or expression A, such that we can prove "there is exactly one n such that A" $(\exists^1 n.A)$, where n is a new name - the defineiendum. If such a unique existence proof exists, we call n *well-defined*. For example, $\forall x.x \notin \Phi$ is an implicit definition for the empty set Φ (instead of explicitly saying $\Phi := \{\}$). We can prove unique existence of Φ by just exhibiting $\{\}$ and showing that any other set S with $\forall x.x \notin S$ we have $S \equiv \Phi$. S cannot have elements, so it has the same elements as Φ, and thus $S \equiv \Phi$.

Here is another implicit definition: The exponential function is that function $f : \mathbb{R} \to \mathbb{R}$ with $f' = f$ and $f(0) = 1$. Here A is the clause "$f' = f$ and $f(0) = 1$". Well-definedness is mathematically non-trivial.

Mathematics uses examples and counterexamples to support understanding a property P. Examples give us a sense of the extent of P and counterexamples help delineate the border of P.

Def 2.14. Example: An example E is a mathematical statement that consists of:

- symbol p for the **exemplandum** (plural: exemplanda): the property to be exemplified,
- the **exemplans** (plural: exemplantia): an expression A denoting a mathematical object that acts as witness object for the property p, and

- (optionally) a **justification** of E, i.e. a proof π that $p(A)$ holds in the current context.

Def 2.15. Counter Example: An example for the complement of the property p where π is a proof that $p(A)$ does not hold.

Def 2.16. Running Example: An example that is recalled time and again during an argument, applying the newly discovered knowledge to it, presumably to show how things work.

For instance, the following mathematical statement is a mathematical example:

The identity function on $\mathbb{R}$ is *continuous*.

- the exemplandum is "*continuous*",
- the exemplans A is "The identity function on $\mathbb{R}$", and
- the justification π, a proof of continuity $\mathrm{Id}_{\mathbb{R}}$: Let $\epsilon > 0$, then we choose $\delta := \epsilon \cdots$ is omitted.

We can use examples for $\forall$-statements as well. Let $B = \forall x.A$ be a mathematical statement, then we call an example/counter example for B whose exemplandum is the property "A holds for x" an example/counter example for B. For example, 3 is an example for the statement $B = \forall n.\mathrm{odd}(n)$.

Note

An example (or even several) for a mathematical statement B do not constitute a proof for B. On the other hand, one counter example for a mathematical statement B does show that $\neg B$ is true ($\exists x.\neg A \equiv \neg(\forall x.A)$).

Def 2.17. Syllogism: A syllogism (proof method) is an argument that applies deductive reasoning to arrive at a conclusion based on two (or more) propositions that are asserted or assumed to be true. A **syllogistic system** is simply a set of syllogisms

Note

Today we usually reserve the term syllogism to informal arguments, formal arguments are called inference rules.

In a **proof by contradiction**, we make an assumption ($\neg A$) to the contrary of what we want to prove (A), and then we show that the assumption leads to a contradiction and therefore must be false. That lets us to conclude that ($\neg\neg A$) must be true, and thus (A).

If we want to prove a statement of the form *"If A, then B"*, then we do that by:

- assuming A and proving B from that.
- after this subproof, we may not use A anymore.

We call this proof method a **proof by local hypothesis**

If we know a general rule of the form *"If A then B"*, and we also know a specific instance A', which arises from A by replacing its variables with concrete values. Then by **chaining** we can conclude the corresponding instance B', obtained from B by the same variable replacement.

Example.

1. We know: "Socrates is a human." This is A'.
2. We also know a general rule:
$$\forall x\ (\ x \text{ is human } \Rightarrow\ x \text{ is mortal }).$$
This is the "If A then B" statement, where
$$A(x) = (x \text{ is human}), \quad B(x) = (x \text{ is mortal}).$$
3. Substitute $x =$ Socrates:
$$A(x) \to \text{Socrates is human}, \qquad B(x) \to \text{Socrates is mortal}.$$
4. Since "Socrates is human" is true, chaining gives us:
$$\text{Socrates is mortal}.$$

If we want to prove *"A for all x"*, then we prove A without regard for x, then argue something like *"as x was chosen arbitrarily when we proved A, we know A for every x"*

If we know that one of the cases $A_1, \cdots, A_k$ must always hold, then we use **proof by cases** to show that *"if A_1 then C"* and "if A_2 then C" and so on $\forall i : 1 \leq i \leq k$, hence we can conclude that C holds. The following is an example:

Lemma 2.1. For all rational numbers a and b, if $ab = 0$, then $a = 0$ or $b = 0$.

Proof.

1. Let $a, b \in \mathbb{Q}$ and assume $ab = 0$.
2. Obviously, either $a = 0$ or $a \neq 0$.
3. We prove that $a = 0$ or $b = 0$ by the two cases induced.
   4. Case $a = 0$:
      4.1 Then the conclusion of the lemma is trivially true, and there is nothing to prove.
   5. Case $a \neq 0$:
      5.1 We can multiply both sides of the equation $ab = 0$ by $\frac{1}{a}$ and obtain
      $$\tfrac{1}{a} \cdot ab = \tfrac{1}{a} \cdot 0.$$
      5.2 Reducing the fractions gives $b = 0$.
6. In both cases, either $a = 0$ or $b = 0$, so the statement holds. □

Def 2.18. Without Loss of Generality (WLOG): An inference principle used in proofs when a statement $Q(x)$ must be shown for all cases of x, but the different cases are equivalent by symmetry or trivial reduction. Formally, if the domain of x can be partitioned into cases $A_1, \ldots, A_k$ and it can be shown that:

- proving $Q(x)$ under one representative case A_i suffices, because
- for every other case A_j there is either a trivial proof of $Q(x)$ or a direct reduction to the representative case,

then we may assume A_i holds "without loss of generality". This allows us to simplify the argument by only proving the representative case.

For example, if we want to prove the property $Q(p)$ for all primes p. We know that every prime is either 2 (the only even prime) or an odd prime. If the case $p = 2$ is easy, and the general odd prime case is the only real work, then we can say:

> WLOG, assume p is odd

This does not exclude the even case; it means we have already checked that the even case adds no new difficulty.

In mathematical vernacular, different alphabets are used to convey precise meanings. Table 1 presents the Greek letters that you are expected to read, recognize, and write. In addition, table 2 shows the curly Roman and Fraktur letters, which are also standard in mathematical writing. Becoming fluent with these alphabets is an essential part of mathematical literacy.

α	A	alpha	β	B	beta	γ	Γ	gamma
δ	Δ	delta	ϵ	E	epsilon	ζ	Z	zeta
η	H	eta	θ, ϑ	Θ	theta	ι	I	iota
κ	K	kappa	λ	Λ	lambda	μ	M	mu
ν	N	nu	ξ	Ξ	xi	o	O	omicron
π, ϖ	Π	pi	ρ	P	rho	σ	Σ	sigma
τ	T	tau	υ	Υ	upsilon	φ, ϕ	Φ	phi
χ	X	chi	ψ	Ψ	psi	ω	Ω	omega

Table 1: Greek Alphabet

Roman	$\mathcal{A}, \mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}, \mathcal{F}, \mathcal{G}, \mathcal{H}, \mathcal{I}, \mathcal{J}, \mathcal{K}, \mathcal{L}, \mathcal{M}, \mathcal{N}, \mathcal{O}, \mathcal{P}, \mathcal{Q}, \mathcal{R}, \mathcal{S}, \mathcal{T}, \mathcal{U}, \mathcal{V}, \mathcal{W}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}$
Fraktur	$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \mathfrak{F}, \mathfrak{G}, \mathfrak{H}, \mathfrak{I}, \mathfrak{J}, \mathfrak{K}, \mathfrak{L}, \mathfrak{M}, \mathfrak{N}, \mathfrak{O}, \mathfrak{P}, \mathfrak{Q}, \mathfrak{R}, \mathfrak{S}, \mathfrak{T}, \mathfrak{U}, \mathfrak{V}, \mathfrak{W}, \mathfrak{X}, \mathfrak{Y}, \mathfrak{Z}$

Table 2: Roman and Fraktur Letters

3 Unary Natural Numbers

Def 3.1. Unary Natural Numbers: Terms generated by the following: $u ::= 0 \mid s(u)$ where 0 denotes zero and $s(u)$ denotes the successor of u. i.e. $\{0, s(0), s(s(0)), s(s(s(0))), \cdots\}$.

Def 3.2. Piano Axioms: The following set of axioms are called the Piano axioms. They can be used to *reason* about natural numbers:

- **P1.** 0 is a unary natural number.
- **P2.** Every unary natural number has a successor that is unary natural number and that is different from it.
- **P3.** 0 is not a successor of any unary natural number.
- **P4.** Different unary natural numbers have different successors.
- **P5. Induction Axiom**. Every unary natural number possess a property P if
  - 0 has the property P, and
  - the successor of every unary natural number that has the property P also possesses property P

More formally, we call the set of unary natural numbers $\mathbb{N}_1$, and we say $P(n)$ if a unary natural number $n \in \mathbb{N}_1$ has a property P. Therefore, we express the axioms as follows:

- $0 \in \mathbb{N}_1$
- $\forall n \in \mathbb{N}_1 . s(n) \in \mathbb{N}_1 \wedge n \neq s(n)$
- $\neg(\exists n \in \mathbb{N}_1 . 0 = s(n))$
- $\forall n \in \mathbb{N}_1 . \forall m \in \mathbb{N}_1 . n \neq m \Rightarrow s(n) \neq s(m)$
- $\forall P . P(0) \wedge (\forall n \in \mathbb{N}_1 . P(n) \Rightarrow P(s(n))) \Rightarrow (\forall m \in \mathbb{N}_1 . P(m))$

Theorem 3.1. Every unary natural number is either zero (0) or the successor of a unary natural number.

Proof. We make use of the induction axiom P5: We use the property P of "being zero or a successor" and prove the statement by convincing ourselves of the prerequisites of:

1. 0 is zero so 0 is "zero or a successor".
2. Let n be arbitrary unary natural number that is "zero or a successor".
3. Then its successor is a "successor", so the successor of n is also "zero or a successor".
4. Since we have taken n arbitrary we have shown that for any n, its successor has property P.
5. Property P holds for all unary natural number by P5, so we have proven the assertion.

□

Def 3.3. Mathematical Induction: A method for proving a statement $P(n)$ is true for every natural number n. A proof by mathematical induction consists of two steps:

- The **base case**: Prove that $P(0)$ is true.
- The **induction step**: Prove that for every n, if $P(n)$ is true then $P(n+1)$ is also true.

The hypothesis in the induction step, that $P(n)$ is true for a particular n, is called the **induction hypothesis**

Def 3.4. The addition "function": We define the addition operation $+$ procedurally:

- Adding 0 to a number does not change it, expressed as: $n + 0 = n$.
- Adding m to the successor of n yields the successor of $m + n$, expressed as: $m + s(n) = s(m + n)$.

Theorem 3.2. For all unary natural number n, m, l, we have $(n + m) + l = n + (m + l)$.

Proof. We prove this by induction on l.

Let $P(l)$ be the property:

$$(n + m) + l = n + (m + l).$$

- base case:

$$(n + m) + 0 = (n + m) = n + (m + 0)$$

- step case: Assume $(n+m)+l = n+(m+l)$ for an arbitrary l. We must show

$$(n+m)+s(l) = n+(m+s(l)).$$

$$\begin{aligned}(n+m)+s(l) &= s\big((n+m)+l\big)\\ &\overset{\text{IH}}{=} s\big(n+(m+l)\big)\\ &= n+s(m+l)\\ &= n+(m+s(l)).\end{aligned}$$

  Hence the property holds for $s(l)$.

□

Def 3.5. Multiplication: The **unary multiplication** operation can be defined by the equations:

- $n \cdot 0 = 0$, and
- $n \cdot s(m) = n + (n \cdot m)$

Def 3.6. Exponentiation: The **unary exponentiation** operation can be defined by the equations:

- $exp(n, 0) = s(0)$, and
- $exp(n, s(m)) = n \cdot exp(n, m)$

Def 3.7. Summation: The **unary summation** operation can be defined by the equations:

- $\bigoplus_{i=0}^{0}(n_i) = 0$, and
- $\bigoplus_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigoplus \bigoplus_{i=0}^{m}(n_i)$

Example:

$$\begin{aligned}\bigoplus_{i=0}^{s(s(0))}(n_i) &= n_{s(s(0))} \bigoplus \bigoplus_{i=0}^{s(0)}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus \bigoplus_{i=0}^{0}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus 0\end{aligned}$$

Def 3.8. Product: The **unary product** operation can be defined by the equations:

- $\bigodot_{i=0}^{0}(n_i) = s(0)$, and
- $\bigodot_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigodot \bigodot_{i=0}^{m}(n_i)$

4 Sets

Def 4.1. Set: A collection of different things (called **elements** or **members** of the set).

We can represent sets by listing the elements within curly brackets, e.g. $\{a, b, c\}$ or by describing the elements via a property, e.g. $\{x \mid x \mod 2 = 0\}$ (the set of even numbers). We can state element-hood ($a \in S$) or not ($b \notin S$)

> Note
>
> Learn to distinguish between objects and their representations. ($\{a, b, c\}$ and $\{b, a, a, c\}$ are different representations of the same set)

Def 4.2. Set Equality: $A \equiv B :\equiv \forall x. x \in A \Leftrightarrow x \in B$

Def 4.3. Subset: $A \subseteq B :\equiv \forall x. x \in A \Rightarrow x \in B$

Def 4.4. Proper Subset: $A \subset B :\equiv (A \subseteq B) \wedge (A \not\equiv B)$

Def 4.5. Super Set: $A \supseteq B :\equiv \forall x. x \in B \Rightarrow x \in A$

Def 4.6. Proper Superset: $A \supset B :\equiv (A \supseteq B) \wedge (A \not\equiv B)$

Def 4.7. Set Union: $A \cup B := \{x \mid x \in A \vee x \in B\}$

Def 4.8. Set Intersection: $A \cap B := \{x \mid x \in A \wedge x \in B\}$

Def 4.9. Disjoint: Two sets A, B are **disjoint** iff $A \cap B = \emptyset$

Def 4.10. Set Difference: $A \setminus B := \{x \mid x \in A \wedge x \notin B\}$

Def 4.11. Power Set: $\mathcal{P}(A) := \{S \mid S \subseteq A\}$ (the set of all subsets)

> Note
>
> Set operators such as union ($\cup$), intersection ($\cap$), and set difference ($\setminus$) always return a *set*.
> For example, consider
> $$\{a, b\} \cap \{b, c\} = \{b\}.$$
> The result $\{b\}$ is a set.
> Now observe:
> - $\{b\} \notin \{a, b\}$, since the elements of $\{a, b\}$ are a and b, not the set $\{b\}$.
> - $\{b\} \notin \{b, c\}$, since the elements of $\{b, c\}$ are b and c, not the set $\{b\}$.
> - $\{b\} \subseteq \{a, b\}$, because every element of $\{b\}$ (namely b) is also in $\{a, b\}$.
> - $\{b\} \subseteq \{b, c\}$, for the same reason.
>
> Thus, the outcome of a set operation should always be compared to other sets using $\subseteq$ (subset relation), not $\in$ (element relation).

Def 4.12. Cartesian Product: $A \times B := \{(a, b) \mid a \in A \wedge b \in B\}$, (a, b) is a *pair*

Def 4.13. Family: A set of sets

Def 4.14. Topology: A family of open sets

Def 4.15. Indexing Function: Let I and X be sets. A function $f : I \to X$ is called an indexing function. The set I is called the index set. For each $i \in I$, we define $x_i := f(i)$, here i is called the index (or parameter) of x_i.

Def 4.16. Indexed Family: Given an indexing function $f : I \to X$, the set of values $f(I) = \{f(i) : i \in I\}$ is called an indexed family. It is often written as $(x_i)_{i \in I}$, $\langle x_i \rangle_{i \in I}$, or $\{x_i\}_{i \in I}$.

Example: Consider index set $I := \{1, 2, 3\}$. Consider indexing function f defined as:

$$f(1) = \{a, b\} \quad f(2) = \{c\} \quad f(3) = \{d, f\}$$

the set of values $f(I) = \{f(1), f(2), f(3)\}$ is the indexed family:

$$(X_i)_{i \in I} = f(I) = \{\{a, b\}, \{c\}, \{d, f\}\}$$

where $x_i = f(i)$, i.e. $x_1 = f(1) = \{a, b\}, \cdots$

Def 4.17. Union over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcup_{i \in I} X_i := \{x \mid \exists i \in I . x \in X_i\}$

Def 4.18. Intersection over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcap_{i \in I} X_i := \{x \mid \forall i \in I . x \in X_i\}$

Def 4.19. n-fold Cartesian Product: $\prod_{i=1}^{n} X_i := X_1 \times \cdots \times X_n := \{\langle x_1, \cdots, x_n \rangle \mid \forall i . 1 \leq i \leq n \Rightarrow x_i \in X_i\}$ where $\langle x_1, \cdots, x_n \rangle$ is called an n-tuple.

Example: consider the indexed family $(X_i)_{i \in \{1,2,3\}} = \{\{a, b\}, \{c\}, \{d, f\}\}$. $X_1 \times X_2 \times X_3 = \{\langle a, c, d \rangle, \langle a, c, f \rangle, \langle b, c, d \rangle, \langle b, c, f \rangle\}$.

Def 4.20. n-dim Cartesian Space: $X^n := \{\langle x_1, \cdots, x_n \rangle \mid 1 \leq i \leq n \Rightarrow x_i \in X\}$ where $\langle x_1, \cdots, x_n \rangle$ is called a vector.

Example: Let $X = \{1, 2\}$, then $X^2 = \{\langle 1, 1 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle, \langle 2, 2 \rangle\}$. Another example is the usual 2D plane: $\mathbb{R}^2 := \{\langle x, y \rangle \mid x \in \mathbb{R}, y \in \mathbb{R}\}$

Def 4.21. Size of a Set: The size $\Gamma(A)$ of a set A is the number of elements in A.

Note

- $\Gamma(A \cup B) \leq \Gamma(A) + \Gamma(B)$
- $\Gamma(A \cap B) \leq \min(\Gamma(A), \Gamma(B))$
- $\Gamma(A \times B) = \Gamma(A) \cdot \Gamma(B)$

Def 4.22. Pairwise Disjoint: A family of sets is called **pairwise disjoint** or **mutually disjoint**, if any two of those sets are disjoint.

Def 4.23. Multiset: A **multiset** (or **bag**) is a generalization of a set where elements can appear more than once. Formally, a multiset $\mathcal{M}$ over a domain D is a pair (D, m), where $m : D \to \mathbb{N}$ is a function mapping each element $x \in D$ to its **multiplicity** (the number of occurrences).

Def 4.24. Multiset Membership: $x \in \mathcal{M} :\equiv m(x) > 0$

Def 4.25. Multiset Size: The size of a multiset $\mathcal{M} = (D, m)$, denoted $\Gamma(\mathcal{M})$, is the sum of the multiplicities of all its elements: $\Gamma(\mathcal{M}) = \sum_{x \in D} m(x)$.

Def 4.26. Multiset Sum (Union): The sum of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \uplus \mathcal{M}_2$, is a multiset where the multiplicity of each element is the sum of its multiplicities: $m_{\uplus}(x) = m_1(x) + m_2(x)$.

Def 4.27. Multiset Intersection: The intersection of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \cap \mathcal{M}_2$, is a multiset where the multiplicity of each element is the minimum of its multiplicities in the two multisets: $m_{\cap}(x) = \min(m_1(x), m_2(x))$.

Def 4.28. Multiset Equality: $\mathcal{M}_1 \equiv \mathcal{M}_2 :\equiv \forall x \in D . m_1(x) = m_2(x)$

5 Relations

Def 5.1. Relation: $R \subseteq A \times B$ is a (binary) relation between A and B

Def 5.2. Relation on: a relation $R \subseteq A \times B$ where $A = B$ is called a *relation on* A

Def 5.3. Total: A relation $R \subseteq A \times B$ is called *total* (*left total*) iff $\forall x \in A. \exists y \in B. (x, y) \in R$

Def 5.4. Converse Relation: $R^{-1} \subseteq B \times A := \{(y, x) \mid (x, y) \in R\}$ is the converse relation of R

Def 5.5. Relation Composition: The composition of $R \subseteq A \times B$ and $S \subseteq B \times C$ is defined as $R \circ S := \{(a, c) \in A \times C \mid \exists b \in B. (a, b)$

A relation on A: $R \subseteq A \times A$ is called:

- **reflexive** on A, iff $\forall a \in A. (a, a) \in R$
- **irreflexive** on A, iff $\forall a \in A. (a, a) \notin R$
- **symmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \in R$
- **asymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \notin R$
- **antisymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \wedge (b, a) \in R \Rightarrow a = b$
- **transitive** on A, iff $\forall a, b, c \in A. (a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$
- **equivalence relation** on A, iff R is reflexive symmetric, and transitive

Def 5.6. Equality Relation: The equality relation is an equivalence relation on any set.

Def 5.7. Divides Relation: The *divides* relation on the integers is defined as $| \subseteq \mathbb{Z} \times \mathbb{Z}$, where $| = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid \exists k \in \mathbb{Z} : b = a \cdot k\}$. We write $a \mid b$ to denote $(a, b) \in |$ (read as *"a divides b"*).

For example: $5 \mid 10$ because $\exists k : 10 = 5 \cdot k$ where $k = 2$.

Def 5.8. Congruence Modulo: For a fixed $n \in \mathbb{N}$ with $n \geq 1$, the *congruence modulo n* relation on the integers is defined as

$$\equiv_n \subseteq \mathbb{Z} \times \mathbb{Z}, \qquad \equiv_n = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid n \mid (a - b)\}.$$

We write $a \equiv b \pmod{n}$ to denote $(a, b) \in \equiv_n$ (read as *"a is congruent to b modulo n"*).

Examples:

- $12 \equiv 2 \pmod{5}$ because $5 \mid 10$
- $7 \equiv 22 \pmod{5}$ because $5 \mid 15$
- $9 \equiv 9 \pmod{5}$ because $5 \mid 0$

Note

In the definition of congruence modulo n, we do *not* take the absolute value of the difference:

$$a \equiv b \pmod{n} \iff n \mid (a - b).$$

Since the divisor n can be multiplied by a negative integer, the sign of $(a - b)$ does not matter. For example, $2 \equiv 12 \pmod{5}$ because $2 - 12 = -10$ and $-10 = 5 \cdot (-2)$, so $5 \mid (2 - 12)$.

Exercise: Which of the following binary relations on integers are equivalence relations?

1. Φ. It is not reflexive because we must have $(a, a) \in R$ for all integers a. But Φ has no pairs at all. $\rightarrow$ NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a, a) \in R \quad \forall a$
   - It is symmetric because if $(a, b) \in R$ then (b, a) also is
   - It is transitive because every possible pair is in it anyway
   - $\rightarrow$ YES.
3. $m \leq n$

- It is reflexive since $m \leq m$
- It is not symmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
- $\rightarrow$ NO

4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ YES.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). However, it is not symmetric ($2 \mid 4$ but $4 \nmid 2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to 2 modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (negative or positive does not matter)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ YES.
7. $m \mid n$. It is reflexive but not symmetric. $\rightarrow$ NO.

Def 5.9. Parital Ordering (non-strict): A relation $R \subseteq A \times A$ is called a **partial ordering** on A, iff R is reflexive, antisymmetric, and transitive on A.

Def 5.10. Strict Partial Ordering: A relation $R \subseteq A \times A$ is called a **strict partial ordering** on A, iff R is irreflexive, and transitive on A.

> **Note**
>
> We usually write strict partial ordering with asymmetric symbols like $\prec$ and non-strict ones by adding a line that reminds us of equality ($\preceq$).

Def 5.11. Comparable: In a non-strict partial ordering, two elements a, b are comparable if either $a \preceq b$ or $b \preceq a$. If neither holds, they are incomparable.

Def 5.12. Linear Order: A partial ordering is called linear (or total) on A, iff all elements in A are comparable, i.e. if $(x, y) \in R$ or $(y, x) \in R$ for all $x, y \in A$.

For example, the $\leq$ relation is a linear order on $\mathbb{N}$. However, the "divides" ($|$) relation is a non-strict partial ordering on $\mathbb{N}$ that is not linear (e.g., neither $2 \mid 3$ nor $3 \mid 2$).

Lemma 5.1. Strict partial orderings are asymmetric.

Proof. By contradiction: if $(a, b) \in R$ and $(b, a) \in R$, then $(a, a) \in R$ by transitivity, and that can't be because its irreflexive. □

Lemma 5.2. If $\preceq$ is a non-strict partial order, then $\prec := \{(a, b) \mid a \preceq b \wedge a \neq b\}$ is a strict partial order. Conversely, if $\prec$ is a strict partial order, then $\preceq := \{(a, b) \mid a \prec b \ \vee \ a = b\}$ is a non-strict partial order.

Proof. **(1) Non-strict $\rightarrow$ Strict:** Assume $\preceq$ is reflexive, antisymmetric, and transitive. Define $a \prec b \iff (a \preceq b \wedge a \neq b)$.

- *Irreflexive:* $a \prec a$ would require $a \neq a$, impossible.
- *Transitive:* If $a \prec b$ and $b \prec c$, then $a \preceq b$, $b \preceq c$, and $a \neq b$, $b \neq c$. By transitivity of $\preceq$, $a \preceq c$. If $a = c$, antisymmetry would force $a = b$, contradiction. So $a \neq c$, hence $a \prec c$.

Thus $\prec$ is a strict partial order.

(2) Strict $\rightarrow$ Non-strict: Assume $\prec$ is irreflexive and transitive. Define $a \preceq b \iff (a = b \vee a \prec b)$.

- *Reflexive:* $a = a$ gives $a \preceq a$.
- *Antisymmetric:* If $a \preceq b$ and $b \preceq a$, then either $a = b$ or we would have $a \prec b$ and $b \prec a$, contradicting asymmetry.
- *Transitive:* If $a \preceq b$ and $b \preceq c$, then combining cases ($=$ or $\prec$) always gives $a \preceq c$.

Thus $\preceq$ is a non-strict partial order. $\square$

Examples:

- On $\mathbb{N}$, the non-strict partial order $\leq$ induces the strict order $<$ by removing equality. Conversely, $<$ induces $\leq$ by adding equality.
- On sets, the non-strict partial order $\subseteq$ induces the strict order $\subset$. Conversely, $\subset$ induces $\subseteq$ by adding equality.

Exercise: which of the following binary relation on integers are partial ordering:

1. Φ. It is not reflexive because we must have $(a, a) \in R$ for all integers a. But Φ has no pairs at all. $\rightarrow$ NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a, a) \in R \quad \forall a$
   - It is symmetric because if $(a, b) \in R$ then (b, a) also is, $\rightarrow$ it is not antisymmetric, $\rightarrow$ NO
3. $m \leq n$
   - It is reflexive since $m \leq m$
   - It is asymmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
   - It is transitive
   - $\rightarrow$ YES
4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ NO.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). It is asymmetric on $\mathbb{N}$ but not on $\mathbb{Z}$ ($2 \mid -2$ and $-2 \mid 2$ but $2 \neq -2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to n modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (not asymmetric)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ NO.
7. $m \mid n$. It is reflexive but not asymmetric. $\rightarrow$ NO.

Def 5.13. Partially Ordered Set (Poset): A **Partially Ordered Set** (or **poset**) is a pair $\langle G, \preceq \rangle$ where G is a set and $\preceq$ is a non-strict partial ordering on G.

Def 5.14. Least Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **least** (or **smallest**, **minimal**, or $\preceq$**-minimal**) element of T iff $t \preceq t'$ for all $t' \in T$.

Def 5.15. Greatest Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **greatest** (or **largest**, **maximal**, or $\preceq$**-maximal**) element of T iff $t' \preceq t$ for all $t' \in T$.

Def 5.16. Supremum and Infimum: Let $\langle G, \preceq \rangle$ be a non-strict partial ordering and $T \subseteq G$, then we call the **least upper bound** ($\sup(T) \in G$) of T the **supremum** of T and the **greatest lower bound** ($\inf(T) \in G$) of T the **infimum** of T (if they exist).

Example: Consider the non-strict partial ordering $\langle \mathbb{R}, \preceq \rangle$. Let T be the open interval $(0, 1) \subseteq \mathbb{R}$, then $\sup(T) = 1$, and $\inf(T) = 0$. Note that $0, 1 \in \mathbb{R}$ but $\notin T$. (its different from `min` and `max` of T).

6 Functions

Def 6.1. Partial function: A relation $R \subseteq X \times Y$ is a *partial function* from X to Y ($f : X \rightharpoonup Y$) iff $\forall x \in X, \forall y_1, y_2 \in Y.\big((x, y_1) \in R \wedge (x, y_2) \in R \Rightarrow y_1 = y_2\big)$ (i.e. there is at most one $y \in Y$ with $(x, y) \in f$)

Def 6.2. dom, codom: We call X the domain ($\text{dom}(f)$), and Y the codomain ($\text{codom}(f)$)

Def 6.3. Function Application: Instead of writing $(x, y) \in f$ we write $f(x) = y$

Def 6.4. Undefined: we call a partial function $f : X \rightharpoonup Y$ undefined at $x \in X$, iff $\forall y \in Y.(x, y) \notin f$ ($f(x) = \perp$).

Def 6.5. Function: A partial function $f : X \rightharpoonup Y$ is called a *function* (or *total function*) from X to Y iff it is a total relation. $(\forall x \in X.\exists^1 y \in Y : (x, y) \in f)$

> Note
>
> A partial function guarantees uniqueness, but not existence. So each x has at most one y (some might have none). A total function guarantees both uniqueness and existence (at most and at least $\rightarrow$ exactly one)

Def 6.6. Partial Function Space: Given two sets X and Y, the set of all possible partial functions from X to Y is called the partial function space, denoted as $X \rightharpoonup Y$

> Note
>
> If X and Y are finite, then for each element $x \in X$ we have $\Gamma(Y) + 1$ choices (either pick one of the $\Gamma(Y)$ values in Y, or leave it undefined). Therefore, $\Gamma(X \rightharpoonup Y) = (\Gamma(Y) + 1)^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightharpoonup Y = \{\Phi, \{(0,0)\}, \{(0,1)\}, \{(1,0)\}, \{(1,1)\}, \{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightharpoonup Y) = 9 = (\Gamma(Y) + 1)^{\Gamma(X)} = (2 + 1)^2$

Def 6.7. Function Space: Given two sets X and Y, the set of all possible functions from X to Y is called the partial function space, denoted as $X \rightarrow Y$

> Note
>
> If X and Y are finite, then for each element $x \in X$ we must pick exactly one of the $\Gamma(Y)$ values in Y. Therefore, $\Gamma(X \rightarrow Y) = (\Gamma(Y))^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightarrow Y = \{\{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightarrow Y) = 4 = (\Gamma(Y))^{\Gamma(X)} = (2)^2$

A function $f : X \rightarrow Y$ is called

- **injective** iff $\forall x_1, x_2 \in X.f(x_1) = f(x_2) \Rightarrow x_1 = x_2$
- **surjective** iff $\forall y \in Y.\exists x \in X.f(x) = y$
- **bijective** iff f is injective and surjective

Let's look for some examples to make the previous definitions clear.

Assume $X = \{1, 2, 3\}$ $Y = \{a, b\}$ The relation $R = \{(1, a), (1, b)\}$ is just a relation, it is not a partial function nor a total function.

If $R = \{(1, a)\}$ it becomes a partial function but not a total one.

If $R = \{(1, a), (2, a), (3, a)\}$ it becomes a total function because each $x \in X$ has exactly one $y \in Y$. However, it is not injective ($1, 2, 3$ all map to the same value) and not surjective (b is not reached).

IF $R = \{(1, a), (2, a), (3, b)\}$ then this is a surjective function but not an injective one (since $1, 2$ map to the same value).

Assume $X = \{1, 2\}$ $Y = \{a, b, c\}$ $f = \{(1, a), (2, b)\}$ then this is an injective function (different inputs map to different output) but not surjective since there is no x such that $f(x) = c$.

And finally, assume $X = \{1, 2\}$ $Y = \{a, b\}$ $f = \{(1, a), (2, b)\}$ then this is a total function that is both injective and surjective, i.e. a bijective function.

Def 6.8. Function Composition: If $f : A \to B$ and $g : B \to C$ are functions, then we call $g \circ f : A \to C; x \longmapsto g(f(x))$ the composition of g and f (read g after f).

> Note
>
> Given $f, g : \mathbb{N} \to \mathbb{N}$, if f and g both are injective/bijective, so is $g \circ f$

Observations:

- If f is injective, then the converse relation f^{-1} is a partial function.

  For example: $X = \{1, 2, 3\}, Y = \{a, b, c, d\}, f : X \to Y = \{(1, a), (2, b), (3, c)\}$. f is clearly injective since different inputs map to different outputs. $\text{dom}(f^{-1}) = \text{codom}(f)$ and vice versa. $f^{-1} = \{(a, 1), (b, 2), (c, 3)\}$. It is a partial function because for $d \in \text{dom}(f^{-1})$ there is no $f^{-1}(d)$

- If f is surjective, then the converse relation f^{-1} is a total relation.

  For example: $X = \{1, 2, 3\}, Y = \{a, b\}, f : X \to Y = \{(1, a), (2, b), (3, b)\}$. f is clearly surjective because both a, b are hit by some x. $f^{-1} = \{(a, 1), (b, 2), (b, 3)\}$. It is a total function.

- IF f is bijective, call the converse relation **inverse function**, we also write it as f^{-1}.

- If f is bijective, then the inverse function f^{-1} is a total function.

- If $f : X \to Y$ is bijective, then $f \circ f^{-1} = \text{Id}_Y$.

  For example: $X = \{1, 2, 3\}, Y = \{a, b, c\}, f = \{(1, a), (2, b), (3, c)\}, f^{-1} = \{(a, 1), (b, 2), (c, 3)\}$. $(f \circ f^{-1})(y) = f(f^{-1}(y))$

  - $y = a : f^{-1}(a) = 1, f(1) = a$
  - $y = b : f^{-1}(b) = 2, f(2) = b$
  - $y = c : f^{-1}(c) = 3, f(3) = c$

  $\to$ $f \circ f^{-1} = \text{Id}_Y$

Def 6.9. Image and Preimage: Let $f : X \to Y$ be a function, $X' \subseteq X$ and $Y' \subseteq Y$, then we call

- $f(X') := \{y \in Y \mid \exists x \in X'.(x, y) \in f\}$ the **image** of X' under f,
- $\text{Im}(f) := f(X)$ the **image** of f, and
- $f^{-1}(Y') := \{x \in X \mid \exists y \in Y'.(x, y) \in f\}$ the **preimage** of Y' under f.

> Note
>
> Let $f : \mathbb{N} \to \mathbb{N}$ be injective and let its partial inverse be
>
> $$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \qquad \text{dom}(f^{-1}) = \text{im}(f), \quad f^{-1}(y) = x \text{ iff } f(x) = y.$$
>
> Then $f^{-1} \circ f = \text{Id}_{\mathbb{N}}$ as a total function $\mathbb{N} \to \mathbb{N}$.
> *Proof.* For any $x \in \mathbb{N}$ we have $f(x) \in \text{im}(f) = \text{dom}(f^{-1})$, hence $(f^{-1} \circ f)(x) = f^{-1}(f(x)) = x$. □

For example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = 2x$. Then f is injective with image $\text{im}(f) = \{0, 2, 4, 6, \dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = \tfrac{y}{2} \text{ if } y \text{ is even}, \quad \text{undefined if } y \text{ is odd}.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(2x) = \tfrac{2x}{2} = x,$$

hence $f^{-1} \circ f = \text{Id}_{\mathbb{N}}$. By contrast, $f \circ f^{-1}$ is only the identity on the even numbers.

Another example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = x + 1$. Then f is injective with image $\text{im}(f) = \{1, 2, 3, \dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = y - 1 \text{ if } y \geq 1, \quad \text{undefined if } y = 0.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(x+1) = (x+1) - 1 = x,$$

hence again $f^{-1} \circ f = \mathrm{Id}_{\mathbb{N}}$. Here $f \circ f^{-1}$ is only the identity on $\{1, 2, 3, \dots\}$.

Def 6.10. Isomorphic: We call two sets A and B isomorphic iff there is a bijection $f : A \to B$.

Def 6.11. Cardinality: We say that a set A is **finite** and has **cardinality** $\Gamma(A) \in \mathbb{N}$, iff there is a bijective function $f : A \to \{n \in \mathbb{N} \mid n < \Gamma(A)\}$.

Def 6.12. Countably Infinite: We say that a set A is countably infinite, iff there is a bijective function $f : A \to \mathbb{N}$. The cardinality of such set is denoted by $\aleph_0$ Examples: $\mathbb{N}, \mathbb{Z}, \cdots$

Def 6.13. Uncountably Infinite: We say that a set A is **uncountably infinite**, iff A is infinite and there is **no** bijective function $f : A \to \mathbb{N}$. The cardinality of the such sets is denoted by $\mathfrak{c}$ (for continuum). Examples: $\mathbb{R}, [0, 1], \mathbb{C}, \mathcal{P}(\mathbb{N})$

Def 6.14. Countable: A set A is called countable, iff it is finite or countably infinite

> Note
>
> The set of real numbers $\mathbb{R}$ is not countable (there is no bijection) $\mathbb{R} \to \mathbb{N}$

Def 6.15. Restriction: Let $f : A \to B$ and $C \subseteq A$, then we call the function $f \restriction_C := \{(c, b) \in f \mid c \in C\}$ the restriction of f to C.

Def 6.16. Lambda-Notation: Let X be a set and E an expression depending on a variable $x \in X$. The expression $\lambda x \in X.\, E$ denotes the (partial) function f from X to Y with $f(x) = E$ for all $x \in X$.

Examples:

- the identity function is written in lambda notation: $\lambda n \in \mathbb{N}.n = \{(n, n) \mid n \in \mathbb{N}\} = \mathrm{Id}_{\mathbb{N}} = \{(0, 0), (1, 1), (2, 2), \cdots\}$
- the square function: $\lambda x \in \mathbb{N}.x^2 = \{(x, x^2) \mid x \in \mathbb{N}\} = \{(0, 0), (1, 1), (2, 4), (3, 9), \cdots\}$
- the "add" operation: $\lambda(x, y) \in \mathbb{N} \times \mathbb{N}.x + y = \{((0, 0), 0), ((1, 0), 1), \cdots\}$

Def 6.17. Curried Function Type: Let X, Y, Z be sets. An *uncurried* function is written as

$$f : X \times Y \to Z, \quad f(x, y) = E(x, y).$$

The same function can equivalently be written in *curried* form as

$$f : X \to Y \to Z, \quad f(x)(y) = E(x, y).$$

Thus in the curried notation the function takes one argument at a time: for $x \in X$, the value $f(x)$ is itself a function $Y \to Z$, and applying it to $y \in Y$ yields $f(x)(y) \in Z$.

Example. The addition function can be written in uncurried or curried form:

$$\mathrm{add} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m, n) = m + n,$$

equivalently: $\mathrm{add} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m)(n) = m + n.$

In the curried form, for fixed $m \in \mathbb{N}$, the expression $\mathrm{add}(m) : \mathbb{N} \to \mathbb{N}$ is the function $n \mapsto m + n$, i.e. the function that adds m to its argument.

Another Example (Trinary)

$$h : \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(m, n, o) = m + n + o.$$

Equivalently, in curried form we can write

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o.$$

In this form, $h(m)$ is a function

$$h(m) : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o,$$

which for fixed m returns the function "add m to the sum of two further arguments."

Def 6.18. Well-Defined Function: Let X, Y be sets. A function declaration $f : X \to Y$ together with a defining expression $E(x)$ is called *well-defined* iff for every $x \in X$:

- the expression $E(x)$ is meaningful (all variables are bound and the operations are valid), and
- the result $E(x)$ lies in Y.

Equivalently: a function is well-defined if its type annotation $f : X \to Y$ is consistent with the expression that defines it.

Checking well-definedness in practice:

The general definition above says that a function declaration must be consistent with its defining expression. Typical ways in which this can fail are:

- **Type mismatch:** the declared type $X \to Y$ does not agree with the actual input/output structure of the definition.
- **Free variables:** the definition uses symbols that are not bound by the input.
- **Partiality:** the definition does not provide an output for all inputs in the domain.

To illustrate, let us fix two functions

$$f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \qquad g : \mathbb{N} \to \mathbb{N},$$

and examine a series of possible definitions of functions h. Each case shows whether the definition is well-defined and why.

A correct curried definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n) = f(n, g(m)).$$

This is well-defined: for each $m \in \mathbb{N}$ we obtain $g(m) \in \mathbb{N}$, so $(n, g(m)) \in \mathbb{N} \times \mathbb{N}$ and $f(n, g(m)) \in \mathbb{N}$.

Incorrect composition

$$g \circ f : \mathbb{N} \to \mathbb{N}.$$

This is not well-defined with the given type. Since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, the composition $g \circ f$ has domain $\mathbb{N} \times \mathbb{N}$, hence $g \circ f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, not $\mathbb{N} \to \mathbb{N}$.

A correct nested definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n, o) = f(n, f(m, o)).$$

This is well-defined: $f(m, o) \in \mathbb{N}$, so $(n, f(m, o)) \in \mathbb{N} \times \mathbb{N}$, and thus $f(n, f(m, o)) \in \mathbb{N}$.

Type mismatch between curried and uncurried

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h = f.$$

This is not well-defined, since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ has a different type from the declared type of h. Mathematically $f(x, y)$ and $f(x)(y)$ can be identified via currying, but in type-theoretic terms they are distinct unless an explicit curry operation is applied.

Free variable problem

$$h : \mathbb{N} \to \mathbb{N}, \quad h(x) = f(x, y).$$

This is not well-defined, since y is not bound. Without fixing y in advance, the right-hand side is not a function of x alone.

Partial definition problem

$$h : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(x, 1) = x.$$

This is not a total function, because the definition only specifies values when the second argument is 1. A function of type $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$ must assign a value for every pair (x, y).

Def 6.19. Invertible: Let A and B be sets and $f : A \to B$ a function, then f is called invertible, iff there is a function $g : B \to A$, such that $f \circ g = \mathrm{Id}_B$. g is called the inverse function (or just inverse) of f and is written as f^{-1}.

Theorem 6.1. A function $f : A \to B$ is invertible, iff it is bijective.

Proof.

1. If f is bijective, then it is invertible:
   - Assume $f : A \to B$ is bijective
   - for each $b \in B$ there exists a unique $a \in A$ such that $f(a) = b$. Define $g : B \to A$ by sending b to this unique a. Therefore:
     1. $g \circ f = \mathrm{Id}_A$, because if you start with $a \in A$, apply f, then g, you get back a.
     2. $f \circ g = \mathrm{Id}_B$, because if you start with $b \in B$, apply g, then f, you get back b.

- Hence, g is the inverse of f. So f is invertible

2. If f is invertible, then it is bijective:

Suppose there exists a function $g : B \to A$ such that $g \circ f = \mathrm{Id}_A$ and $f \circ g = \mathrm{Id}_B$. We want to show f is bijective.

1. injectivity
   (a) Assume $f(a) = f(a')$.
   (b) Apply g to both sides:
   $$a = \mathrm{Id}_A(a) = g(f(a)) = g(f(a')) = a'$$
   (c) So, $a = a'$. Hence f is injective.
2. surjectivity
   (a) Take any $b \in B$. Then
   $$f(g(b)) = (f \circ g)(b) = \mathrm{Id}_B(b) = b$$
   (b) So b is indeed an image of f. Hence f is surjective.

□

Lemma 6.2. Let A and B be sets, then the set $\mathcal{R} = \mathcal{P}(A \times B)$ of all relations between A and B and the function space $A \to \mathcal{P}(B)$ are isomorphic.

Proof. For each $R \in \mathcal{R}$ and each $x \in A$ we define: $S_x^R := \{\, y \in B \mid (x, y) \in R \,\}$ (the set of all y in B that are related to x by R). Example:

$$A = \{1, 2\} \quad B = \{a, b, c\} \quad R = \{(1, a), (1, c), (2, b)\}$$

Then for $x = 1 : S_1^R = \{a, c\}$ and for $x = 2 : S_2^R = \{b\}$.

We define a the following mapping:

$$\varphi : \mathcal{R} \to (A \to \mathcal{P}(B)), \quad \varphi(R) = f_R \qquad \text{where } f_R(x) = S_x^R.$$

For our example, this would be $\varphi(R) = f_R = \{1 \mapsto \{a, c\}, 2 \mapsto \{b\}\}$

Then, for any $f \in A \to \mathcal{P}(B)$, we compute the inverse as $\varphi^{-1}(f) = \{(x, y) \mid y \in f(x)\}$. This is exactly the relation corresponding to f.

Since both directions match up perfectly, φ is a bijection, hence an isomorphism. □

Example: the $>$ relation on $\mathbb{N}$ can also be represented as the function $f_>$ that maps a number n to the set $\{n + 1, \cdots\}$, i.e. $f_> : \mathbb{N} \to \mathcal{P}(\mathbb{N})\ ;\ n \mapsto \{\, m \mid m > n \,\}$

Another example: let $A = \{x, y\}, B = \{1\}$, the cartesian product $A \times B = \{(x, 1), (y, 1)\}$. Hence, the power set (all possible relations) is:

$$\mathcal{P}(A \times B) = \{\{\phi\}, \{(x, 1)\}, \{(y, 1)\}, \{(x, 1), (y, 1)\}\}$$

Note: the cardinality of the power set of the cartesian product of two sets A, B is $\Gamma(\mathcal{P}(A \times B)) = 2^{\Gamma(A)\Gamma(B)}$.

$\mathcal{P}(B) = \{\{\phi\}, \{1\}\}$, function space $A \to \mathcal{P}(B)$ is the set of all functions $f : A \to \mathcal{P}(B)$:

$$A \to \mathcal{P}(B) = \{f_1, f_2, f_3, f_4\}$$

where:

- $f_1 = \{(x, \{\phi\}), (y, \{\phi\})\}$
- $f_2 = \{(x, \{\phi\}), (y, \{1\})\}$
- $f_3 = \{(x, \{1\}), (y, \{\phi\})\}$
- $f_4 = \{(x, (1)), (y, \{1\})\}$

Since $\Gamma(A \to \mathcal{P}(B)) = \Gamma(\mathcal{P}(A \times B))$, we can define a bijection:

- $\varphi(\{\phi\}) \mapsto f_1$
- $\varphi(\{(x, 1)\}) \mapsto f_3$
- $\varphi(\{(y, 1)\}) \mapsto f_2$

- $\varphi(\{(x,1),(y,1)\}) \mapsto f_4$

$\rightsquigarrow$ isomorphism.

Note

The bijection $\phi(R) = f$ entails the following properties of R:
- R is a total relation, iff $\phi \notin \operatorname{Im}(f)$
- R is a partial function, iff $\Gamma(S) \leq 1 \quad \forall S \in \operatorname{Im}(f)$
- $R = \phi$, iff $\operatorname{Im}(f) = \{\phi\}$
- R is reflexive, iff $x \in f(x) \quad \forall x \in A$

We have defined equivalence relations as reflexive, symmetric, and transitive relations, e.g., equality is an equivalence relation.

Let S be a set of persons, and $=_A \subseteq S^2$, such that $s =_A t$, iff s and t have the same age, then $=_A$ is an equivalence relation on S.

Let S and T be sets and $S \sim_\Gamma T$, iff there is a bijection $f : S \to T$, then $\sim_\Gamma$ is an equivalence relation. For example, let $S = \{1,2,3\} \quad T = \{a,b,c\}$. We can define a bijection $f = \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$, so, $S \sim_\Gamma T$ (they have the same size).

Lemma 6.3. If S is a set and $R \subseteq S^2$ is an equivalence relation, then $= \subseteq R$.

Proof Sketch: because of reflexivity ($\forall x \in S, (x,x) \in R$), the equality relation $=$ is exactly $\{(x,x) \mid x \in S\}$. Therefore, $= \subseteq R$.

7 Equivalence Classes

Def 7.1. Equivalence Classes and Quotients: Let S be a set and R be an equivalence relation on S, then for any $x \in S$ we call the set $[x]_R := \{\, y \in S \mid R(x,y) \,\}$ the **equivalence class** of x under R, and the set $S/R := \{\, [x]_R \mid x \in S \,\}$ the **quotient space** of S under R, it is often read as S *modulo* R. The element x is called the **representative** of $[x]_R \in S/R$.

Example: Let $S = \mathbb{Z}$ and R is the equivalence relation on S that describes the same parity (congruence modulo 2) $x \equiv y \pmod 2 \Leftrightarrow 2 \mid (x-y)$:

$$R = \{\, (x,y) \in \mathbb{Z} \times \mathbb{Z} \mid x \equiv y \pmod 2 \,\}$$

- $(x,y) \in R$ if the difference $x - y$ is divisable by 2.
- i.e. either both are even, or both are odd.

All odd-odd pairs:

$$\{\, (x,y) \mid x,y \in \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\} \,\}$$

All even-even pairs:

$$\{\, (x,y) \mid x,y \in \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\} \,\}$$

Example pairs in R: $(1,3), (5,-1), (7,7), (2,4), (0,-6), (-8,10)$ Then, we have the following two distinct classes:

- $[1]_R = \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\}$
- $[2]_R = \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\}$

For each $x \in \mathbb{Z}$ we have:

- the set $[1]_R$ as the equivalence class if x is odd
- the set $[2]_R$ as the equivalence class if x is even
- 1 and 2 are the representatives

And the quotient space set of $\mathbb{Z}$ under R is:

$$\mathbb{Z}/R = \{[1]_R, [2]_R\} = \{\text{odds}, \text{evens}\}$$

Def 7.2. Canonical Projection: The mapping $\pi_R : S \to S/R \quad ; \quad x \mapsto [x]_R$ is called the **canonical projection** or **canonical surjection** of S to S/R.

Given our example, we define the following canonical projection:

$$\pi_R : \mathbb{Z} \to \{[1]_R, [2]_R\}, \quad x \mapsto [1]_R \text{ if odd, and } x \mapsto [2]_R \text{ if even.}$$

Def 7.3. System of Representatives: Let R be an equivalence relation on S, a subset $M \subseteq S$ is called a system of representatives, iff M contains exactly one representative for each equivalence class of R.

Given our example, a system of representatives is any subset of $\mathbb{Z}$ that picks exactly one element from each equivalence class, e.g.

- $M = \{1, 2\}$ is a system of representatives
- $M = \{-3, 0\}$ is a system of representatives
- $M = \{5, -8\}$ is a system of representatives

Observation: Often a quotient space S/R behaves similarly to the original set S.

Example: $n \equiv_k m \Leftrightarrow k \mid (n-m)$ is an equivalence relation and $\pi_{\equiv_k} : \mathbb{Z}/\equiv_k \to \{\, n \mid 0 \leq n \leq (k-1) \,\}$ is bijective.

Another example, the general congruence modulo $n \equiv_k m \Leftrightarrow k \mid (n-m)$ partitions $\mathbb{Z}$ into exactly k equivalence classes (all numbers that leave the same remainder when divided by k). e.g. $k = 3$:

- $[0] = \{\cdots, -6, -3, 0, 3, 6, \cdots\}$
- $[1] = \{\cdots, -5, -2, 1, 4, 7, \cdots\}$
- $[2] = \{\cdots, -4, -1, 2, 5, 8, \cdots\}$

We have the canonical projection $\pi_{\equiv_k} : \mathbb{Z} \to \mathbb{Z}/\equiv_k, \quad n \mapsto [n]$ that sends each $n \in \mathbb{Z}$ to its equivalence class.

Notice that the quotient space $\mathbb{Z}/\equiv_k$ has exactly k classes, so, we have the following bijection:

$$\varphi : \mathbb{Z}/\equiv_k \to \{0, 1, \cdots, k-1\}, \quad [n] \mapsto \text{ the remainder of } n \bmod k.$$

If we compose the canonical projection with the bijection we get the remainder, e.g. for $k = 3$:

$$-6 \xmapsto{\pi_{\equiv_3}} [0] \xmapsto{\varphi} 0.$$

$$7 \xmapsto{\pi_{\equiv_3}} [1] \xmapsto{\varphi} 1.$$

Lemma 7.1. Let $=\subseteq S \times S$ be the equality relation on a set S, then $[x]_= \in S/=$ is always a **singleton** and thus $S \sim_\Gamma S/=$.

Proof. If we take the equality relation on a set S, then for any $x \in S$, the equivalence class is:

$$[x]_= = \{\, y \in S \mid y = x \,\}$$

But the only y that satisfies $y = x$ is $y = x$ itself. So we get $[x]_= = \{x\}$. This is a **singleton** set. So every equivalence class under $=$ has exactly one element.

The quotient space is $S/= \;:= \{\, [x]_= \mid x \in S \,\}$ and each $[x]_=$ is the singleton set $\{x\}$, so the quotient space becomes $S/= \;:= \{\, \{x\} \mid x \in S \,\}$. In other words, the quotient space under the equality relation just replaces each element x with one-element set $\{x\}$.

That means the set S is in bijection with its quotient space under $=$ via the following bijective map:

$$\phi : S \to S/=, \quad x \mapsto [x]_= = \{x\}$$

Therefore, we express the bijection via same cardinality:

$$S \sim_\Gamma S/=$$

□

Fun Example: Let S be the set of all cities worldwide and $\sim_c \in S^2$ defined by $a \sim_c b$, iff a and b are in the same country. Then $\sim_c$ is an equivalence relation on S, and the quotient space $S/\sim_c$ of S under $\sim_c$ consists of the equivalence class $[x]_{\sim_c}$ where $x \in S$. The capital of a country is a good representative for an equivalence class.

For instance: let's take a subset of all worldwide cities for simplicity, consider domain of discourse:

$$S = \{\text{Berlin}, \text{Munich}, \text{Hamburg}, \text{Paris}, \text{Lyon}, \text{Marseille}, \text{Rome}, \text{Milan}\}.$$

For $a, b \in S$ define the relation:

$$a \sim_c b \iff a \text{ and } b \text{ are in the same country.}$$

This relation is:

- reflexive: any city is in the same country as itself.
- symmetric: if Berlin is in the same country as Munich, then Munich is in the same country as Berlin.
- transitive: if Berlin and Munich are in the same country, and Munich and Hamburg are in the same country, then Berlin and Hamburg are in the same country.

Hence, $\sim_c$ is an equivalence relation.

The quotient space $S/\sim_c$ is the set of equivalence classes, i.e. the partition of S into countries:

$$[\text{Berlin}]_{\sim_c} = \{\text{Berlin}, \text{Munich}, \text{Hamburg}\}, \quad [\text{Paris}]_{\sim_c} = \{\text{Paris}, \text{Lyon}, \text{Marseille}\}, \quad [\text{Rome}]_{\sim_c} = \{\text{Rome}, \text{Milan}\}.$$

So

$$S/\sim_c = \Big\{\{\text{Berlin}, \text{Munich}, \text{Hamburg}\},\ \{\text{Paris}, \text{Lyon}, \text{Marseille}\},\ \{\text{Rome}, \text{Milan}\}\Big\}.$$

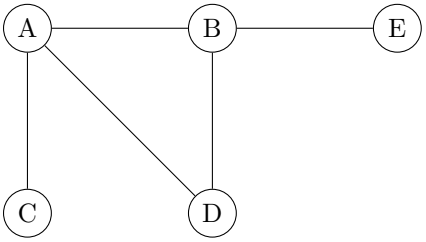
The set of chosen representatives is: $\{\text{Berlin}, \text{Paris}, \text{Rome}\}$.

8 Graphs and Trees

Def 8.1. Undirected Graph: An undirected graph is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq \{\, \{v, \bar{v}\} \mid v, \bar{v} \in V \wedge v \neq \bar{v} \,\}$ is the set of its **undirected edges**.

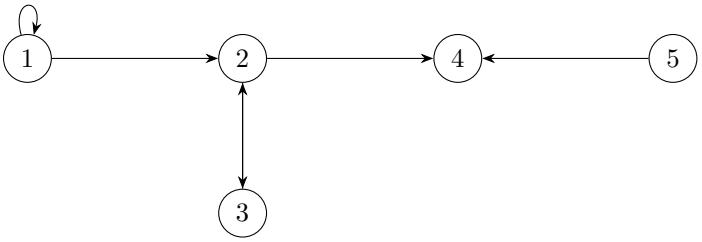
Example: An undirected graph $G_1 = \langle V_1, E_1\rangle$, where $V_1 = \{A, B, C, D, E\}$ and $E_1 = \{\{A, B\}, \{A, C\}, \{A, D\}, \{B, D\}, \{B, E\}\}$



Def 8.2. Directed Graph: A directed graph (digraph) is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq V \times V$ is the set of its **directed edges**.

Example: A directed graph $G_2 = \langle V_2, E_2\rangle$, where $V_2 = \{1, 2, 3, 4, 5\}$ and $E_2 = \{(1, 1), (1, 2), (2, 3), (3, 2), (2, 4), (5, 4)\}$



Def 8.3. Indegree: Given a directed graph $\langle V, E\rangle$, the indegree $\text{indeg}(v)$ of a vertex $v \in V$ is defined as $\text{indeg}(v) = \Gamma(\{\, w \mid (w, v) \in E \,\})$.

Example: $\text{indeg}(2) = 2$

Def 8.4. Outdegree: Given a directed graph $\langle V, E\rangle$, the outdegree $\text{outdeg}(v)$ (branching factor) of a vertex $v \in V$ is defined as $\text{outdeg}(v) = \Gamma(\{\, w \mid (v, w) \in E \,\})$.

Example: $\text{outdeg}(5) = 1$

> Note
>
> For an undirected graph, $\text{indeg}(v) = \text{outdeg}(v)$ for all $v \in V$.

Observation: directed graphs are nothing else than relations.

Def 8.5. Initial vs Terminal Node: Let $G = \langle V, E\rangle$ be a directed graph, then we call a node $v \in V$

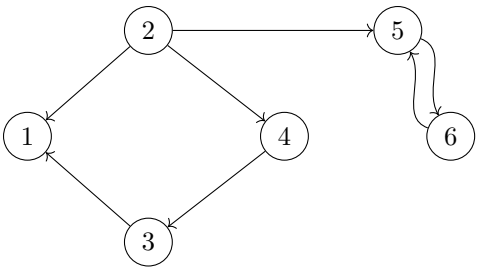
- **initial** (source of G), iff there is no $w \in V$ such that $(w, v) \in E$ (no predecessor)
- **terminal** (sink of G), iff there is no $w \in V$ such that $(v, w) \in E$ (no successor)

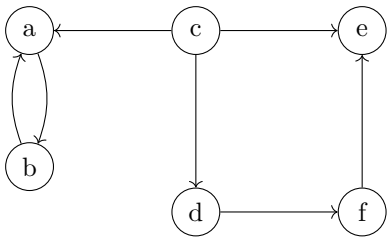
Def 8.6. Graph Isomorphism: Iff we can find a bijection $\psi : V \to \bar{V}$ between two graphs $G = \langle V, E\rangle$ and $\bar{G} = \langle \bar{V}, \bar{E}\rangle$ then we call them isomorphic. The bijection $\psi : V \to \bar{V}$ is defined as $(a, b) \in E \Leftrightarrow (\psi(a), \psi(b)) \in \bar{E}$ for directed graph. And for undirected graph it is defined as $\{a, b\} \in E \Leftrightarrow \{\psi(a), \psi(b)\} \in \bar{E}$

Def 8.7. Equivalent Graphs: Two graphs G and $\bar{G}$ are **equivalent** iff there is an isomorphism ψ between G and $\bar{G}$.

Example: the following two graphs are equivalent as there exists an isomorphism between them given by the bijection:

$$\psi := \{a \mapsto 5, b \mapsto 6, c \mapsto 2, d \mapsto 4, e \mapsto 1, f \mapsto 3\}$$

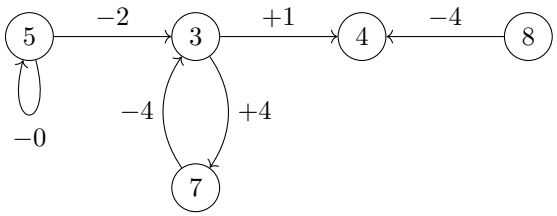




Def 8.8. Labeled Graph: A labeled graph G is a quadruple $\langle V, E, L, l\rangle$ where $\langle V, E\rangle$ is a graph and $l : V \cup E \rightharpoonup L$ is a partial function into a set L of labels.

Example: $G = \langle V, E, L, l\rangle$ with $V = \{A, B, C, D, E\}$, where

- $E = \{(A, A), (A, B), (B, C), (C, B), (B, D), (E, D)\}$
- $l : V \cup E \rightharpoonup \{+, -, \phi\} \times \{1, \cdots, 9\}$ with
  - $l(A) = 5 \quad l(B) = 3 \quad l(C) = 7 \quad l(D) = 4 \quad l(E) = 8,$
  - $l((A, A)) = -0 \quad l((A, B)) = -2 \quad l((B, C)) = +4,$
  - $l((C, B)) = -4 \quad l((B, D)) = +1 \quad l((E, D)) = -4$



Def 8.9. Paths in Graphs: Given a graph $G := \langle V, E\rangle$ we call a $n+1$-tuple $p = \langle v_0, \cdots, v_n\rangle \in V^{n+1}$ a **path** in G iff $(v_{i-1}, v_i) \in E$ for all $1 \leq i \leq n$ and $n > 0$.

- We say that v_i are nodes on p and that v_0 and v_n are **linked** by p.
- v_0 and v_n are called the **start** and **end** of p (write start(p) and end(p)), the other v_i are called **inner nodes** of p.
- n is called the **length** of p (write len(p)).
- We denote the set of paths in G with $\Pi(G)$.

Note

Not all v_i-s in a path are necessarily different.

For a graph $G = \langle V, E\rangle$ and a path $p = \langle v_1, \cdots, v_n\rangle \in V^{n+1}$, write

- $v \in p$, iff $v \in V$ is a vertex on the path ($\exists i.v_i = v$)
- $e \in p$, iff $e = (v, \bar{v}) \in E$ is an edge on the path ($\exists i.v_i = v \wedge v_{i+1} = \bar{v}$)

Example: Consider $G = \langle V, E\rangle$ with $V = \{a, b, c, d\}$ and $E = \{(a, b), (b, c)\}$:

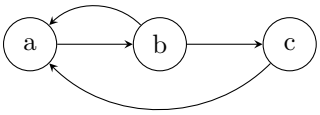
An example path would be $p = \langle a, b, c\rangle$. Here $n = 2$ and $p \in V^3 = \{(a, a, a), (a, b, d), \cdots\}$. Obviously, since $n = 2$ so we have 2 edges and 3 nodes.

- nodes a, b, c are on the path
- start(p) $= a$ and end(p) $= c$
- b is an inner node, and len(p) $= 2$
- $e = (a, b), \bar{e} = (b, c) \in p$
- $\Pi(G) = \{(a, b), (b, c), (a, b, c)\}$

Def 8.10. Cyclic Graphs: Given a directed graph $G = \langle V, E\rangle$, a path p is called **cyclic** iff $\mathrm{start}(p) = \mathrm{end}(p)$. A cycle $\langle v_0, \cdots, v_n\rangle$ is called **simple**, iff $v_i \neq v_j$ for $1 \leq i, j \leq n$ with $i \neq j$ (all inner nodes are distinct).

Def 8.11. DAG: A directed graph with no cycles is called **directed acyclic graph (DAG)**.

Example: $p = \langle a, b, c, a\rangle$ is simple cyclic path. $p = \langle a, b, a, b, a\rangle$ is a cyclic path that is not simple.



Def 8.12. Node Depth: Let $G = \langle V, E\rangle$ be a directed graph, then the depth $\mathrm{dp}(v)$ of a vertex $v \in V$ is defined to be 0, iff v is a source of G and the supremum $\sup(\{\,\mathrm{len}(p) \mid \mathrm{indeg}(\mathrm{start}(p)) = 0 \wedge \mathrm{end}(p) = v\,\})$ otherwise, i.e. the length of the longest path from a source of G to v (can be infinite).

Def 8.13. Graph Depth: Given a directed graph $G = \langle V, E\rangle$, the **depth** ($\mathrm{dp}(G)$) of G is defined as the supremum $\sup(\{\,\mathrm{len}(p) \mid p \in \Pi(G)\,\})$, i.e. the maximal path length in G.

Def 8.14. Tree: A tree is a DAG $G = \langle V, E\rangle$ such that

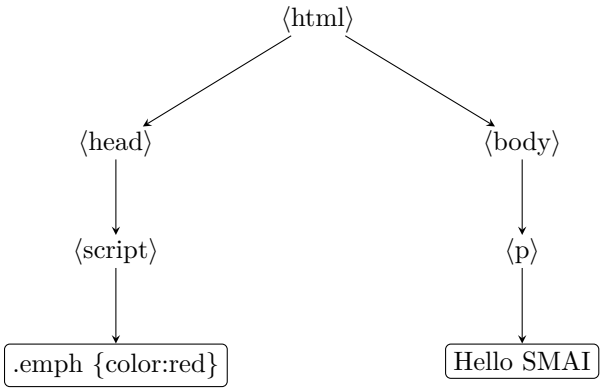
- There is exactly one initial node $v_r \in V$ (called the **root**)
- All nodes but the root have indegree of 1.

We call v the **parent** of w, iff $(v, w) \in E$ (w is a child of v). We call a node v a **leaf** of G, iff it is terminal, i.e. if it does not have children.

Lemma 8.1. For any node $v \in V$ except the root v_r, there is exactly one path $p \in \Pi(G)$ with $\mathrm{start}(p) = v_r$ and $\mathrm{end}(p) = v$.

Observation: We often deal with **well-bracketed structures** in Computer Science, e.g. mathematical expressions, Markup languages like `HTML`, programming languages like `python`, etc. A common approach to scaling in computer science is to come up with a common data structure that allows to program the same algorithm for all of these structures, that where trees come in. For example (`HTML`):

```
<html>
  <head>
    <script>.emph {color:red}</script>
  </head>
  <body>
    <p>Hello SMAI</p>
  </body>
</html>
```



Observation: functional programming languages like SML are ideal for computing with labeled trees.

Example: data types for node labeled trees (binary and general):

```
datatype lbt = leaf of int |t of int * lbt * lbt
```

```
datatype ltree = leaf of int | t of int * ltree list
```

We can compute e.g. the weight (sum of all labels) recursively:

```
fun wt (leaf n) = n | wt (t n l r) = n + wt l + wt r
```

```
fun wt (leaf n) = n
  | wt (t n ts) = n + wtl ts
and wtl (nil) = 0
  | wtl (h::ts) = wt h + wtl ts
```

Observation: often, DAGs are better representations of trees (trees with sharing):

Here, we assume that the constant symbol a represent a leaf and the function symbol f represents a parent node with two children which can be f or a. Hence, the full tree is represented as:

$$f(f(f(a,a),f(a,a)),f(f(a,a),f(a,a)))$$

The DAG compresses the same structure by reusing subterms. It internally does not duplicate all the $f(a,a)$ pieces, it stores them once and reuses them by reference. For example, define a base piece: $t_1 = f(a,a)$, then reuse it: $t_2 = f(t_1,t_1)$ and finally $t_3 = f(t_2,t_2)$

Idea: We want to represent a DAG in SML, and then be able to *explode* it into a tree. A DAG can share subtrees via references and save memory, but a tree has no sharing and every reference is copied. *Exploding* is the process of removing and duplicating the structure.

Datatype Definition

```
type id = int
type label = string

datatype DAG =
   leaf of id * label
 | ref of id
 | dag of id * label * DAG list
```

Example:

```
ex1 =
  dag 1 "f" [
    dag 2 "f" [
      dag 3 "f" [leaf 4 "a", ref 4], ref 3
    ],
    ref 2
  ]
```

Notice that the innermost part is `dag 3 "f" [leaf 4 "a", ref 4]`. That is a node with `id=3`, label "f", and two children: `leaf 4 "a"` (a leaf node with `id=4`, label = "a"), and `ref 4` (a reference back to the same leaf id (4)). So this subtree is $f(a,a)$ but written with sharing.

Next level, we see `dag 2 "f" [(dag 3 ...), ref 3]`. That is a node with id=2, label "f", and two children: one is the whole `dag 3` subtree ($f(a,a)$), the other one is a reference back to it. So this is $f(f(a,a),f(a,a))$.

Finally, we have `dag 1 "f" [(dag 2 ...), ref 2]`. This is the root node with id=1, label "f", and two children. One is the whole `dag 2` ($f(f(a,a),f(a,a))$), the other one is a reference back to that node. So this is the full

$$f(f(f(a,a),f(a,a)),f(f(a,a),f(a,a)))$$

Now, we want a function `fun explode (dag, dict) = tree` where `dict` is a dictionary mapping identifiers to already-exploded subtrees, so that references can be looked up.

Case 1: `explode (leaf (_, l), _) = leaf l`, i.e. if we see a leaf, ignore the id, just return a tree leaf with the label.

Case 2: `explode (ref i, dict) = lookup dict i`, i.e. if we see a reference, we just look up the corresponding tree in the dictionary.

Case 3: internal DAG node:

```
explode (dag (_, l) ds, dict) =
  let val (ts, _) = explist ds dict
  in tree l ts
  end
```

i.e. for a node, explode the children `ds` using `explist` and build a new tree node with label `l` and explode children `ts`

`explist` is a helper function defined as:

```
and explist [] dict = ([], dict)
  | explist (h::t) dict =
      let val (th, dict1) = explode h dict
          val (tt, dict2) = explist t dict1
      in (th::tt, dict2)
      end
```

It explodes a list of DAG children into a list of tree children.

And here are some helper functions for completeness: a function that looks up identifiers:

```
fun did (leaf i _) = i | did (ref _ j) = j | did (dag i _ _) = i
```

A dictionary type and a lookup function:

```
type dict = (id * DAG) list

exception NotFound

fun lookup i [] = raise NotFound
  | lookup i (h::t) =
      let
        val (k,d) = t
      in
        if i = k then d else lookup i t
      end
```

9 Mathematical Structures

Def 9.1. Mathematical Structure: A mathematical structure combines multiple mathematical objects (the components) into a new object. The components usually have names by which they can be referenced. Given a definition of a mathematical structure S, we say that any object that conforms to that is an instance of S.

Example: A graph $G = \langle V, E \rangle$ is a mathematical structure where the components are: V (the set of vertices) and E (the set of edges). A graph G where $V = \{1, 2, 3\}$ and $E = \{(1, 2), (2, 3)\}$ is one instance of a graph.

Def 9.2. Structure Signature: A structure signature combines the components, their types, and accessor names of a mathematical structure in a tabular overview.

Example:

$$\left\langle \begin{array}{lll} V & \text{Set} & \text{vertices (nodes)} \\ E & \{\,(u,v) \mid u, v \in V\,\} & \text{edges} \end{array} \right\rangle$$

> **Note**
>
> Most programming languages have some way of creating *"named structures"* and referencing components is usually done via *"dot notation"* (e.g. `person.name`).

Def 9.3. Group: A group is a mathematical structure $\langle G, \circ, e, .^{-1} \rangle$ that consists of:

- a base set G of objects,
- an operation $\circ : G \times G \to G$, such that $\forall a, b \in G. a \circ b \in G$ and $\forall a, b, c \in G. (a \circ b) \circ c = a \circ (b \circ c)$,
- a unit $e \in G$, such that $\forall a \in G. e \circ a = a \circ e = a$, and
- the inverse function $.^{-1} : G \to G$, such that $\forall a \in G. a \circ a^{-1} = a^{-1} \circ a = e$

An example of a group is the set $G = \mathbb{Z}$, with the operation $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$; then $e = 0$ and $.^{-1}$ is $\lambda x \in \mathbb{Z}. -x$. We write it as $\langle \mathbb{Z}, +, 0, - \rangle$.

Question: can we do the same for multiplication where $e = 1$? If we think about it, we would be stuck on finding the $.^{-1}$ component of the group. What can we always multiply with any integer and get $e = 1$ back? Take for example 1, we can multiply it with $\frac{1}{1}$ (great!). Take -1, we can multiply it with $\frac{1}{-1}$ (also great!). But take any other integer, e.g. 2, we must multiply it with $\frac{1}{2}$ to get 1 back, however, $\frac{1}{2} \notin \mathbb{Z}$, it is however $\in \mathbb{Q}$! Moreover, 0 never has a multiplicative inverse in any field! The multiplicative group would only make sense iff $G = \mathbb{Q} \setminus \{0\}$. We write that as $\langle \mathbb{Q} \setminus \{0\}, *, 1, a \mapsto \frac{1}{a} \rangle$.

But we need some multiplication structure that works for $\mathbb{Z}$!

Def 9.4. Monoid: A monoid is a mathematical structure $\langle M, \circ, e \rangle$ that consists of:

- A base set M of objects,
- An operation $\circ : M \times M \to M$, such that for all $a, b \in M$, we have $a \circ b \in M$ (closure),
- Associativity: for all $a, b, c \in M$, $(a \circ b) \circ c = a \circ (b \circ c)$, and
- A unit element $e \in M$, such that for all $a \in M. e \circ a = a \circ e = a$.

Hence, the mathematical structure $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid (a group missing an inverse function).

We saw that $\langle \mathbb{Z}, +, 0, - \rangle$ is a group under addition. And $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid under multiplication. Now, we want to do everyday arithmetics with integers; we want expressions where we have additions and multiplication (e.g. $a * (b + c)$) which evaluates to $(a * b) + (a * c)$.

For that, we need a mathematical structure where both addition and multiplication live together.

Def 9.5. Ring: A ring is a mathematical structure $\langle R, +, 0, -, *, 1 \rangle$ consisting of:

- A base set R.
- An addition operation $+ : R \times R \to R$ such that $\langle R, +, 0, - \rangle$ is a group (called abelian group).
- A multiplication operation $* : R \times R \to R$ such that $\langle R, *, 1 \rangle$ is a monoid.
- Distributivity: for all $a, b, c \in R, a * (b + c) = (a * b) + (a * c), (a + b) * c = (a * c) + (b * c)$

Hence, mixing components from the group $\langle \mathbb{Z}, +, 0, - \rangle$ and the monoid $\langle \mathbb{Z}, *, 1 \rangle$ leaves us with a ring $\langle \mathbb{Z}, +, 0, -, *, 1 \rangle$

Def 9.6. Magma: A magma is a mathematical structure $\langle M, \circ \rangle$ consisting of:

- A base set M, and
- A binary operation $\circ : M \times M \to M$.

Example: $\langle \mathbb{Z}, - \rangle$ (integers under subtraction). $\langle \mathbb{N}, + \rangle$ (natural numbres under addition)

Def 9.7. Semigroup: A semigroup (magma with associativity) is a mathematical structure $\langle S, \circ \rangle$ consisting of:

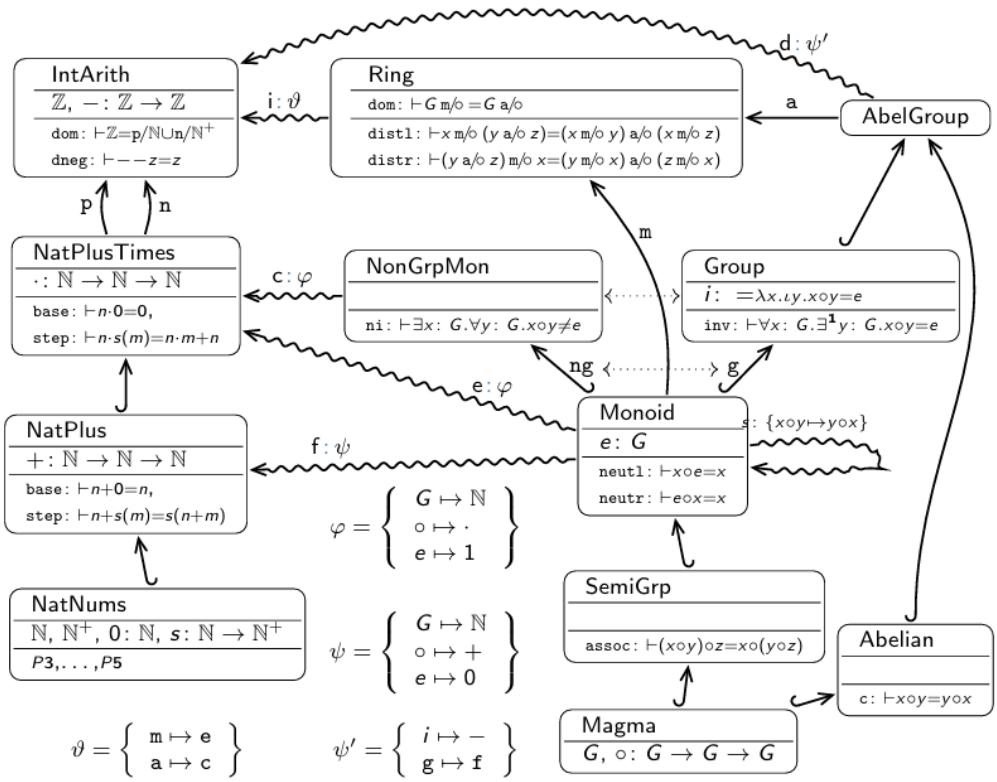
- A base set S,
- A binary opertation $\circ : S \times S \to S$, and
- Associativity: for all $a, b, c \in S, (a \circ b) \circ c = a \circ (b \circ c)$

Example: $\langle \mathbb{Z}, + \rangle$ (integers under addition).

Note

Given collections M and S of instances of magmas and semigroups, $S \subseteq M$ as there are magmas whose operation is not associative.

Mathematical Structures in a Graph:



- nodes are structures with objects ($\circ : \tau$) and properties ($n :\vdash P$).
- edges represent inheritance
  - regular inheritance: $\hookrightarrow$
  - copying substructures: $\xrightarrow{n}$
  - interpretation via φ: $\overset{n}{\rightsquigarrow}$
- Examples are just interpretations ($\rightsquigarrow$) read backwards
- Any object / property / theorem can be copied along any edge

10 Formal Languages and Grammars

Def 10.1. Alphabet: An alphabet is a finite set; we call each element $a \in A$ a **character**, and an n-tuple $s \in A^n$ a **string** (of *length* n over A).

> Note
>
> We often write a string $\langle c_1, \cdots, c_n \rangle$ as "$c_1 \cdots c_n$", for instance `"abc"` for $\langle a, b, c \rangle$

Example: Let $A = \{0, 1\}$ be an alphabet

- `"0"`, `"1"` are strings of length 1 ($A^1 = \{0, 1\}$)
- `"00","01","10","11"` are strings of length 2 in (A^2)
- `"001"` $\in A^3$

Def 10.2. Empty String: Let A be an alphabet, then $A^0 = \{\langle\rangle\}$, where $\langle\rangle$ is the unique 0-tuple. We consider $\langle\rangle$ as the string of length 0 and call it the **empty string** and denote it with ϵ.

> Note
>
> $\{1, 2, 3\} = \{3, 2, 1\}$ BUT $\langle 1, 2, 3 \rangle \neq \langle 3, 2, 1 \rangle$ (sets vs strings)

Def 10.3. String Length: Given a string s, we denote its length with $|s|$.

Def 10.4. String Concatenation: The concatenation $\text{conc}(s, t)$ of two strings $s = \langle s_1, \cdots, s_n \rangle \in A^n$ and $t = \langle t_1, \cdots, t_m \rangle \in A^m$ is defined as $s + t$ or simply `st`

Example: conc("text", "book") = "text" + "book" = "textbook".

Def 10.5. Kleene Plus and Kleene Star: Let A be an alphabet, then we define the sets:

- $A^+ := \bigcup_{i \in \mathbb{N}^+} A^i$ (*nonempty strings*), and
- $A^* := A^+ \cup \{\epsilon\}$ (*strings*).

Def 10.6. Formal Language: A set $L \subseteq A^*$ is called a *formal language* over A.

Example: If $A = \{a, b\}$ then:

$$A^* = \bigcup_{n=0}^{\infty} A^n$$

where $A^0 = \epsilon$, $A^1 = \{a, b\}$, $A^2 = \{a, b, ab, ba\}$, and so on ...

$$A^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$
$$A^+ = \{a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$

A formal language might be $L = \{a, b, aab, bbb, \cdots\}$

Def 10.7. Repeating Chars: We use $c^{[n]}$ for the string that consists of the character `c` repeated n times.

Examples:

- Let $A = \{a, b\}$, $a^{[5]} =$ `"aaaaa"`.
- The set $M := \left\{ \text{ba}^{[n]} \mid n \in \mathbb{N} \right\}$ of strings that start with character b followd by an arbitrary numbers of a's is a formal language over $A = \{a, b\}$

> Note
>
> A formal language might be infinite and even undecidable even if the alphabet A is finite. For example, let $A = \{a, b\}$ then the formal language $L = \{a, ab, bba\}$ is finite but the formal language $L = \{\, a^{[n]} \mid n \in \mathbb{N} \,\}$ is infinite.

Since we cannot list all strings in a formal language L because there are infinitely many, we need a **finite description** that can generate all strings in L. We need **Grammars**.

Def 10.8. Phrase Structure Grammar: A phrase structure grammar (*type 0 grammar, unrestricted grammar, grammar*) is a tuple $\langle N, \Sigma, P, S\rangle$ where

- N is a finite set of **nonterminal symbols**,
- Σ is a finite set of **terminal symbols**. (members of $N \cup \Sigma$ are called symbols).
- P is a finite set of **production rules**: pairs $p := h \to b$ (also written as $h \Rightarrow b$), where $h \in (\Sigma \cup N)^* N (\Sigma \cup N)^*$ and $b \in (\Sigma \cup N)^*$. The string h is called the **head** of p and b the **body**.
- $S \in N$ is a distinguished symbol called the **start symbol** (also **sentence symbol**).

The sets N and Σ are assumed to be disjoint. Any word $w \in \Sigma^*$ is called a **terminal word**.

> Note
>
> Production rules map strings with at least one nonterminal to arbitrary other strings

Notation: If we have n production rule $h \to b_i$ sharing a head, we often write $h \to b_1 \mid \cdots \mid b_n$ instead.

Example: A simple grammar for english sentences:

$$\begin{aligned} \text{S} &::= \text{NP Vi} \\ \text{NP} &::= \text{Article N} \\ \text{Article} &::= \text{the} \mid \text{a} \mid \text{an} \\ \text{N} &::= \text{dog} \mid \text{teacher} \mid \cdots \\ \text{Vi} &::= \text{sleeps} \mid \text{smells} \mid \cdots \end{aligned}$$

Here S is the start symbol, NP, Article, N, and Vi are nonterminals, the, a, dog, smells, ... are terminals. Let's generate a valid sentence from the grammar:

```
NP           Vi
Article N    Vi
the     N    Vi
the    dog   Vi
the    dog sleeps
```

Def 10.9. Lexical: A production rule whose head is a single nonterminal and whose body consists of a single terminal is called **lexical** or a **lexical insertion rule**.

Example: Both of the following rules are lexical

$$\begin{aligned} \text{Noun} &\to \text{"dog"} \\ \text{Verb} &\to \text{"run"} \end{aligned}$$

Def 10.10. Lexicon of a Grammar: The subset of lexical rules of a grammar G is called the **lexicon** of G

Def 10.11. Vocabulary: The set of body symbols are called the vocabulary (or the alphabet).

Def 10.12. Lexical Categories of a Grammar: The nonterminals that appear in the heads of lexical rules are called lexical categories of the grammar.

Def 10.13. Structural Rules: The non lexicon production rules are called structural.

Def 10.14. Phrasal categories: The nonterminals in the heads of production rules that expand into other nonterminals are called phrasal (syntactic) categories.

Def 10.15. G-Derivation: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say G **derives** $t \in (\Sigma \cup N)^*$ from $s \in (\Sigma \cup N)^*$ in one step, iff there is a production rule $p \in P$ with $p = h \to b$ and there are $u, v \in (\Sigma \cup N)^*$, such that $s = uhv$ and $t = ubv$. We write $s \to_G^p t$ (or $s \to_G t$ if p is clear from the context) and use $\to_G^*$ for the reflexive transitive closure of $\to_G$. We call $s \to_G^* t$ a G-derivation of t from s.

Let's unravel that. Take the following grammar:

- $N = \{S\}$

- $\Sigma = \{a, b\}$
- $P = \{S \to \text{aSb}, S \to \text{ab}\}$
- $S = S$

The following derivations holds:

- $S \to_G^p \text{aSb}$ where $p : S \to \text{aSb}$ (*one step*)
- $\text{aSb} \to_G^q \text{aabb}$ where $q : S \to \text{ab}$ (*one step*)
- $S \to_G^{S \to \text{aSb}} \text{aSb} \to_G^{S \to \text{ab}} \text{aabb}$ (*multiple steps*). So, $S \to_G^* \text{aabb}$

Def 10.16. Sentential Form: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say that $s \in (\Sigma \cup N)^*$ is a **sentential form** of G, iff $S \to_G^* s$

Def 10.17. Sentence: A sentential form that does not contain nonterminals is called a sentence of G, we also say that G **accepts** s. We say that G **rejects** s, iff it is not a sentence of G.

Examples

1. "Article teacher Vi" is a sentential form

$$\begin{aligned} S &\to_G \text{NP Vi} \\ &\to_G \text{Article N Vi} \\ &\to_G \text{Article teacher Vi} \end{aligned}$$

2. "the teacher sleeps" is a sentence

$$\begin{aligned} S &\to_G^* \text{Article teacher Vi} \\ &\to_G \text{the teacher Vi} \\ &\to_G \text{the teacher sleeps} \end{aligned}$$

Def 10.18. Language Generation: The language $L(G)$ of G is the set of its sentences. We say that $L(G)$ is **generated** by G.

Def 10.19. Equivalent Grammars: We call two grammars **equivalent**, iff they have the same languages.

Def 10.20. Universal Grammar: A grammar G is said to be **universal** if $L(G) = \Sigma^*$

Def 10.21. Syntactic Analysis: Syntactic analysis (parsing / syntax analysis) is the process of analyzing a string of symbols, either in a formal or a natural language by means of a grammar.

> **Note**
>
> The shape of the grammar determines the size of its language

Def 10.22. Context-Sensitive: We call a grammar context-sensitive (or type 1), if the bodies of production rules have no less symbols than the heads.

Example: The language $\{a^{[n]}b^{[n]}c^{[n]}\}$ is accepted by

$$\begin{aligned} S &\to \text{a b c} \mid A \\ A &\to \text{a } A\ B \text{ c} \mid \text{a b c} \\ \text{c } B &\to B \text{ c} \\ \text{b } B &\to \text{b b} \end{aligned}$$

Def 10.23. Context-Free: We call a grammar context-free (or type 2), iff the heads have exactly one symbol.

Example: The language $\{a^{[n]}b^{[n]}\}$ is accepted by $S \to \text{ a } S \text{ b } \mid \epsilon$

Def 10.24. Regular: We call a grammar regular (or type 3, or REG), if additionally the bodies are empty or consist of a nonterminal, optionally followed by a terminal symbol.

Example: The language $\{a^{[n]}\}$ is accepted by $S \to S \text{ a } \mid \epsilon$

Note

By extension, a formal language L is called *context-sensitive / context-free / regular*, iff it is the language of a respective grammar. Context-free grammars are sometimes `CFGs` and context-free languages `CFLs`.

Observation: Natural languages are probably context sensitive but parsable in real time!

11 Probability Theory

Def 11.1. Sample Space: We say that a set Ω is a **sample space** of a random experiment, iff Ω contains all possible outcomes of that experiment. Each individual outcome $\omega \in \Omega$ is called a **sample point** or an **elementary outcome**.

Example 11.1. Throwing two dice $\rightsquigarrow \Omega = \{\langle 1,1\rangle, \langle 1,2\rangle, \cdots, \langle 6,6\rangle\}$ with $\Gamma(\Omega) = 36$

Example 11.2. Number of tosses until the first head $\rightsquigarrow \Omega = \{1,2,3,\cdots\} = \mathbb{N}$ with $\Gamma(\Omega) = \aleph_0$

Example 11.3. A random point in the interval $[0,1] \rightsquigarrow \Omega = [0,1]$ with $\Gamma(\Omega) = \mathfrak{c}$

Def 11.2. Event: Given a sample space Ω, we say that E is an **event** iff $E \subseteq \Omega$. That is, an event is a subset of the sample space. We say an event E occurs iff the outcome ω of the experiment satisfies $\omega \in E$.

Note

Because events are sets, we use Set Theory to describe relations between them:
- Complement $\bar{E}$: the event E that does not occur
- Union $A \cup B$: the event that A or B or both occurs
- Intersection $A \cap B$: the event that both A and B occur
- Disjoint / Mutually Exclusive: two events A and B are disjoint iff $A \cap B = \emptyset$

Example 11.4. The sample space for rolling one die is $\Omega = \{1,2,\cdots,6\}$, table 3 shows some events of interest.

Event Description	**Set Representation**
A: Even Number	$\{2,4,6\}$
B: Prime Number	$\{2,3,5\}$
$A \cap B$ (Even and Prime)	$\{2\}$
$A \cup B$ (Even or Prime)	$\{2,3,4,5,6\}$
$\bar{A}$ (Not Even)	$\{1,3,5\}$

Table 3: Examples of Events

Def 11.3. σ-algebra: Given a sample space Ω, a collection of subsets $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ is called a **σ-algebra** iff it satisfies the following three axioms:

1. **Non-emptiness:** The sample space is included: $\Omega \in \mathcal{F}$. (which implies $\emptyset \in \mathcal{F}$ via complementation)
2. **Closure under complementation:** If $A \in \mathcal{F}$, then $\bar{A} \in \mathcal{F}$.
3. **Closure under countable unions:** If $A_1, A_2, \cdots \in \mathcal{F}$ is a sequence of events, then $\bigcup_{i=1}^{\infty} A_i \in \mathcal{F}$.

The pair $(\Omega, \mathcal{F})$ is called a **measurable space**.

Example 11.5. Let the sample space be $\Omega = \{1,2,3\}$.

1. Consider the collection $\mathcal{F} = \{\emptyset, \{1\}, \{2,3\}, \Omega\}$. This is a σ-algebra because:
- $\Omega \in \mathcal{F}$ (and $\emptyset \in \mathcal{F}$).
- It is closed under complements: $\bar{\{1\}} = \{2,3\} \in \mathcal{F}$ and $\bar{\emptyset} = \Omega \in \mathcal{F}$.
- It is closed under unions: $\{1\} \cup \{2,3\} = \Omega \in \mathcal{F}$.

2. Consider the collection $\mathcal{G} = \{\emptyset, \Omega, \{1\}, \{2\}\}$. This fails to be a σ-algebra for two reasons:
- No closure under Complements: $\{1\} \in \mathcal{G}$, but its complement $\bar{\{1\}} = \{2,3\}$ is not in $\mathcal{G}$.
- No closure under Unions: $\{1\} \in \mathcal{G}$ and $\{2\} \in \mathcal{G}$, but their union $\{1,2\}$ is not in $\mathcal{G}$.

Def 11.4. Probability Measure: A **probability measure** P is a function $P : \mathcal{F} \to [0,1]$ that assigns a real number to every event $A \in \mathcal{F}$, satisfying the following **Kolmogorov Axioms**:

1. **Non-negativity:** $P(A) \geq 0$ for all $A \in \mathcal{F}$.
2. **Normalization:** $P(\Omega) = 1$.

3. **Countable Additivity:** For any sequence of disjoint events $A_1, A_2, \cdots \in \mathcal{F}$:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Def 11.5. Probability Space: A **probability space** is the triplet $\langle \Omega, \mathcal{F}, P \rangle$, where:

- Ω is the sample space.
- $\mathcal{F}$ is a σ-algebra on Ω.
- P is a probability measure defined on $(\Omega, \mathcal{F})$.

Example 11.6. For a fair 6-sided die roll, the probability space is defined as follows:

- $\Omega = \{1, 2, 3, 4, 5, 6\}$
- $\mathcal{F} = \mathcal{P}(\Omega)$ (the power set)
- $P(A) = \frac{\Gamma(A)}{\Gamma(\Omega)} = \frac{\Gamma(A)}{6}$

Note

From the Kolmogorov Axioms, we can derive the following fundamental properties of probability measures:

- **Complement Rule:** For any event A, $P(\bar{A}) = 1 - P(A)$.
- **Monotonicity:** If $A \subseteq B$, then $P(A) \leq P(B)$.
- **Inclusion-Exclusion Principle:** For any two events A and B:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **Set Partition Identity:** For any two events A and B, B can be split into parts that are in A and parts that are not:

$$P(B) = P(A \cap B) + P(\bar{A} \cap B)$$

- **Impossible Event:** $P(\emptyset) = 0$.

Def 11.6. Random Variable: Given a probability space $\langle \Omega, \mathcal{F}, P \rangle$, a **random variable** X is a function $X : \Omega \to D$ that maps each sample point $\omega \in \Omega$ to a numerical value in a set $D \subseteq \mathbb{R}$. We call D the **domain** of X, denoted by $\text{dom}(X)$.

Example 11.7. In a coin flip experiment where $\Omega = \{\text{Head}, \text{Tail}\}$, we can define a random variable X such that:

- $X(\text{Head}) = 1$
- $X(\text{Tail}) = 0$

In this case, $\text{dom}(X) = \{0, 1\}$.

Def 11.7. Probability of a Random Variable: For a random variable X and a specific value $x \in \text{dom}(X)$, the probability that X takes the value x is defined as the measure of the event consisting of all outcomes ω that map to x:

$$P(X = x) := P\left(\{\omega \in \Omega \mid X(\omega) = x\}\right)$$

Example 11.8. Consider rolling a fair die where $\Omega = \{1, 2, \ldots, 6\}$. If I define a random variable Y as the square of the outcome, then the probability of the event $Y = 4$ is:

$$P(Y = 4) = P\left(\left\{\omega \in \Omega \mid \omega^2 = 4\right\}\right) = P\left(\{2\}\right) = \frac{1}{6}$$

Def 11.8. Probability of Negation: For a random variable X and $x \in \text{dom}(X)$, we define the probability of the logical negation as:

$$P(X \neq x) := P(\{\omega \in \Omega \mid X(\omega) \neq x\}) = 1 - P(X = x)$$

Def 11.9. Probability of Disjunction: For random variables X_1, X_2, we define the probability of the logical disjunction ("or") as:

$$P(X_1 = x_1 \vee X_2 = x_2) := P(\{\omega \in \Omega \mid X_1(\omega) = x_1\} \cup \{\omega \in \Omega \mid X_2(\omega) = x_2\})$$

Def 11.10. Joint Probability: Given two random variables X and Y defined on the same sample space Ω, the **joint probability** of $x \in \text{dom}(X)$ and $y \in \text{dom}(Y)$ is defined as the probability of the intersection of their individual events:

$$P(X = x, Y = y) := P(\{\omega \in \Omega \mid X(\omega) = x\} \cap \{\omega \in \Omega \mid Y(\omega) = y\})$$

Example 11.9. In a roll of two dice where X is the first die and Y is the second die: The joint probability $P(X = 3, Y = 4)$ is the probability of the sample point $(3, 4) \in \Omega$:

$$P(X = 3, Y = 4) := P(\{(3, 4)\}) := \frac{1}{36}$$

Example 11.10. To find the probability that the sum of two dice is 7 but neither is 3, we define the following random variables on the sample space $\Omega = \{1, \ldots, 6\}^2$:

- X_1, X_2: outcomes of the first and second die respectively.
- S: the sum, where $S(\omega) = X_1(\omega) + X_2(\omega)$.

We are interested in the event E:

$$E = \{\omega \in \Omega \mid S(\omega) = 7\} \cap \{\omega \in \Omega \mid X_1(\omega) \neq 3\} \cap \{\omega \in \Omega \mid X_2(\omega) \neq 3\}$$

Step 1: Identify outcomes where $S = 7$:

$$A = \{(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)\}$$

Step 2: Apply the constraints $X_1 \neq 3$ and $X_2 \neq 3$:

- $\omega \in A$ and $X_1(\omega) = 3 \implies \omega = (3, 4)$
- $\omega \in A$ and $X_2(\omega) = 3 \implies \omega = (4, 3)$

Step 3: Calculate the cardinality of the filtered set:

$$E = A \setminus \{(3, 4), (4, 3)\} = \{(1, 6), (2, 5), (5, 2), (6, 1)\} \implies \Gamma(E) = 4$$

Step 4: Use the probability measure:

$$P(E) = \frac{\Gamma(E)}{\Gamma(\Omega)} = \frac{4}{36} = \frac{1}{9}$$

Def 11.11. Marginalization: For two random variables X and Y, the **marginal probability** of X is calculated by summing the joint probabilities over all $y \in \text{dom}(Y)$:

$$P(X = x) := \sum_{y \in \text{dom}(Y)} P(X = x, Y = y)$$

This is a direct application of the axiom of countable additivity to the partition of the event $\{X = x\}$ into smaller, mutually exclusive joint events.

Example 11.11. Suppose I have a joint probability table for two random variables X and Y representing a simplified weather/activity model:

	$Y = 0$ (Stay)	$Y = 1$ (Walk)	**Total**
$X = 0$ (Rain)	0.3	0.1	$P(X = 0) = \mathbf{0.4}$
$X = 1$ (Sun)	0.1	0.5	$P(X = 1) = \mathbf{0.6}$

Table 4: Marginalization Example

To find $P(X = 0)$, I marginalize over Y:

$$P(X = 0) = P(X = 0, Y = 0) + P(X = 0, Y = 1) = 0.3 + 0.1 = 0.4$$

Def 11.12. Conditional Probability: Let A and B be events in a probability space where $P(B) > 0$. The **conditional probability** of A given B is defined as:

$$P(A \mid B) := \frac{P(A \cap B)}{P(B)}$$

In this context, we refer to:

- $P(A)$ as the **prior** probability.
- $P(B)$ as the **evidence**.
- $P(A \mid B)$ as the **posterior** probability.

Example 11.12. Suppose I roll a fair 6-sided die, so $\Omega = \{1, 2, 3, 4, 5, 6\}$. Let A be the event that the outcome is 2, and B be the event that the outcome is even.

- $P(A) = P(\{2\}) = 1/6$
- $P(B) = P(\{2, 4, 6\}) = 3/6 = 1/2$
- $P(A \cap B) = P(\{2\}) = 1/6$

The probability that I roll a 2 **given** I know the result is even is:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{1/6}{1/2} = \frac{1}{3}$$

Theorem 11.1. $P(A \mid C) = 1 - P(\bar{A} \mid C)$

Proof.

1. Partition: $C = (A \cap C) \cup (\bar{A} \cap C)$.
2. Note that $(A \cap C)$ and $(\bar{A} \cap C)$ are disjoint because $A \cap \bar{A} = \emptyset$.
3. Apply the Axiom of Additivity: $P(C) = P(A \cap C) + P(\bar{A} \cap C)$.
4. Rearrange: $P(\bar{A} \cap C) = P(C) - P(A \cap C)$.
5. Conditional probability:
$$P(\bar{A} \mid C) = \frac{P(\bar{A} \cap C)}{P(C)}$$
6. Substitute:
$$P(\bar{A} \mid C) = \frac{P(C) - P(A \cap C)}{P(C)}$$
7. Simplify:
$$P(\bar{A} \mid C) = \frac{P(C)}{P(C)} - \frac{P(A \cap C)}{P(C)} = 1 - P(A \mid C)$$
8. Rearrange the terms:
$$P(A \mid C) = 1 - P(\bar{A} \mid C)$$

□

Theorem 11.2. $P(A \mid C) = P(A \cap B \mid C) + P(A \cap \bar{B} \mid C)$

Proof.

1. Partition: $(A \cap C) = (A \cap C \cap B) \cup (A \cap C \cap \bar{B})$
2. Note that $(A \cap C \cap B)$ and $(A \cap C \cap \bar{B})$ are disjoint because $B \cap \bar{B} = \emptyset$.
3. Axiom of Additivity: $P(A \cap C) = P(A \cap B \cap C) + P(A \cap \bar{B} \cap C)$
4. Conditional probability:
$$P(A \mid C) = \frac{P(A \cap C)}{P(C)}$$

5. Substitute:
$$P(A \mid C) = \frac{P(A \cap B \cap C) + P(A \cap \bar{B} \cap C)}{P(C)}$$

6. Distribute:
$$P(A \mid C) = \frac{P(A \cap B \cap C)}{P(C)} + \frac{P(A \cap \bar{B} \cap C)}{P(C)}$$

7. Finally:
$$P(A \mid C) = P(A \cap B \mid C) + P(A \cap \bar{B} \mid C)$$

□

Theorem 11.3. $P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$

Proof.

1. Conditional probability
$$P(A \cup B \mid C) = \frac{P((A \cup B) \cap C)}{P(C)}$$

2. Distribute: $(A \cup B) \cap C = (A \cap C) \cup (B \cap C)$

3. Inclusion-Exclusion:
$$P((A \cap C) \cup (B \cap C)) = P(A \cap C) + P(B \cap C) - P((A \cap C) \cap (B \cap C))$$

4. Simplify: $(A \cap C) \cap (B \cap C) = A \cap B \cap C$

5. Substitute:
$$P(A \cup B \mid C) = \frac{P(A \cap C) + P(B \cap C) - P(A \cap B \cap C)}{P(C)}$$

6. Simplify:
$$P(A \cup B \mid C) = \frac{P(A \cap C)}{P(C)} + \frac{P(B \cap C)}{P(C)} - \frac{P(A \cap B \cap C)}{P(C)}$$

7. Finally:
$$P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$$

□

Theorem 11.4.
$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} \qquad \textbf{(Bayes' Theorem)}$$

Proof.

1. conditional probability:
$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

2. Multiply both sides by $P(B)$ to express the joint probability:
$$P(A \cap B) = P(A \mid B) \cdot P(B)$$

3. conditional probability:
$$P(B \mid A) = \frac{P(B \cap A)}{P(A)}$$

4. Multiply both sides by $P(A)$ to get a second expression for the joint probability:
$$P(B \cap A) = P(B \mid A) \cdot P(A)$$

5. intersection is commutative $\rightsquigarrow P(A \cap B) = P(B \cap A)$
6. Equate the right-hand sides of the expressions from Step 2 and Step 4:

$$P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

7. Divide both sides by $P(B)$ (assuming $P(B) > 0$) to isolate the **posterior** probability:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

□

Def 11.13. Bayesian Terminology: Within the context of Bayes' Theorem, we define the following nomenclature:

- **Posterior:** $P(A \mid B)$ is the probability of the hypothesis A after observing the evidence B.
- **Likelihood:** $P(B \mid A)$ is the probability that the evidence B would be observed if the hypothesis A were true.
- **Prior:** $P(A)$ is the probability of the hypothesis A before the evidence B is taken into account.
- **Evidence:** $P(B)$ is the total probability of observing the evidence under all possible hypotheses.

Def 11.14. The Product Rule: For any two events A and B where the conditional probabilities are well-defined ($P(A) > 0, P(B) > 0$), the probability of their joint occurrence is:

$$P(A \cap B) = P(A \mid B)P(B)$$
$$P(A \cap B) = P(B \mid A)P(A)$$

This rule establishes that the probability of "A and B" is the probability of one event, scaled by the probability of the other event occurring given that the first has occurred.

Example 11.13. Suppose I draw two cards from a deck without replacement. Let A be the event the first card is an Ace, and B be the event the second card is an Ace.

- $P(A) = 4/52$
- $P(B \mid A) = 3/51$ (since one Ace is gone)

The probability of drawing two Aces is:

$$P(A \cap B) = P(B \mid A)P(A) = \frac{3}{51} \cdot \frac{4}{52}$$

Def 11.15. Independence: Two events A and B are **independent** iff $P(A \cap B) = P(A) \cdot P(B)$

Theorem 11.5. If A and B are independent, then the posterior equals the prior:

$$P(A \mid B) = P(A) \quad \text{and} \quad P(B \mid A) = P(B)$$

Proof.

1. Conditional probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

2. Independence:

$$P(A \mid B) = \frac{P(A) \cdot P(B)}{P(B)} = P(A)$$

3. The proof for $P(B \mid A) = P(B)$ follows identical logic by symmetry.

□

Def 11.16. Pairwise Independence: A collection of events $A_1, A_2, \ldots, A_n$ is **pairwise independent** iff every pair of distinct events is independent. That is, for all $i \neq j$:

$$P(A_i \cap A_j) = P(A_i) \cdot P(A_j)$$

Def 11.17. Mutual Independence: A collection of events $A_1, A_2, \ldots, A_n$ is **mutually independent** iff for every finite subset of indices $I \subseteq \{1, 2, \ldots, n\}$, the probability of their intersection is the product of their individual probabilities:

$$P\left(\bigcap_{i \in I} A_i\right) = \prod_{i \in I} P(A_i)$$

Note

Mutual independence strictly implies pairwise independence, but pairwise independence does not guarantee mutual independence.

Def 11.18. Conditional Independence: Given events A, B, C with $P(C) \neq 0$, then A and B are **conditionally independent** given C iff $P(A \mid B \cap C) := P(A \mid C)$, or $P(B \mid A \cap C) := P(B \mid C)$:

Theorem 11.6. If A and B are conditionally independent given C, then:

$$P(A \cap B \mid C) := P(A \mid C) \cdot P(B \mid C)$$

Proof.

1. By definition:
$$P(A \cap B \mid C) = \frac{P(A \cap B \cap C)}{P(C)}$$
2. By the product rule:
$$P(A \cap B \cap C) = P(A \mid B \cap C) \cdot P(B \cap C)$$
3. Expanding $P(B \cap C)$ further:
$$P(A \cap B \cap C) = P(A \mid B \cap C) \cdot P(B \mid C) \cdot P(C)$$
4. Substituting Step 3 into Step 1:
$$P(A \cap B \mid C) = \frac{P(A \mid B \cap C) \cdot P(B \mid C) \cdot P(C)}{P(C)}$$
5. Simplifying:
$$P(A \cap B \mid C) = P(A \mid B \cap C) \cdot P(B \mid C)$$
6. Since A and B are conditionally independent given C, $P(A \mid B \cap C) = P(A \mid C)$, yielding:
$$P(A \cap B \mid C) = P(A \mid C) \cdot P(B \mid C)$$

□

Def 11.19. Probability Distribution: A **probability distribution** is a vector v with values in $[0, 1]$ such that $\sum_i v_i = 1$.

Note

The **probability distribution** of a random variable X is a complete description of the probabilities $P(X = x)$ for all values $x \in \text{dom}(X)$. It completely characterizes the behavior of the random variable by mapping each possible outcome to its corresponding probability, effectively describing how the total probability of 1 (from the Normalization axiom) is distributed across the domain of X.

Def 11.20. Full Joint Probability Distribution: Given a set of random variables $X_1, X_2, \ldots, X_n$, a **full joint probability distribution** is a probability distribution that assigns a probability to every possible, mutually exclusive joint probability assignment for all variables. It can be represented as an n-dimensional tensor where each entry $P(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) \in [0, 1]$ corresponds to a single atomic event, and the sum over all possible combinations of values is 1:

$$\sum_{x_1 \in \text{dom}(X_1)} \cdots \sum_{x_n \in \text{dom}(X_n)} P(X_1 = x_1, \ldots, X_n = x_n) = 1$$

Example 11.14. Consider two boolean random variables: W (Weather: Sunny=1, Rainy=0) and C (Commute: Walk=1, Drive=0). The full joint probability distribution is given by the table:

	$C = \text{Walk}$	$C = \text{Drive}$	**Total**
$W = \text{Sunny}$	0.6	0.1	0.7
$W = \text{Rainy}$	0.05	0.25	0.3
Total	0.65	0.35	**1.0**

Note that the sum of all internal cells ($0.6 + 0.1 + 0.05 + 0.25 = 1.0$) satisfies the normalization requirement.

Def 11.21. Conditional Probability Distribution: Given random variables X and Y, the **conditional probability distribution** $\mathbf{P}(X \mid Y)$ represents a complete table or matrix of probabilities, providing the distribution of X for every possible observed value $y \in \text{dom}(Y)$. It consists of the conditional probabilities $P(X = x \mid Y = y)$ for all $x \in \text{dom}(X)$ and all $y \in \text{dom}(Y)$. For any fixed y, the distribution over X satisfies the normalization constraint:

$$\sum_{x \in \text{dom}(X)} P(X = x \mid Y = y) = 1$$

Example 11.15. Continuing from the previous example, we can form the conditional probability distribution $\mathbf{P}(C \mid W)$ by dividing each joint probability by the marginal probability of W. Note that for any fixed value of W (each column), the probabilities sum to 1:

$\mathbf{P}(C \mid W)$	$W = \text{Sunny}$	$W = \text{Rainy}$
$C = \text{Walk}$	$0.6/0.7 = 6/7$	$0.05/0.3 = 1/6$
$C = \text{Drive}$	$0.1/0.7 = 1/7$	$0.25/0.3 = 5/6$
Total	**1.0**	**1.0**